

Complete Application Package

Document Contents

1. Self-Hosting Guide (setup, HA YAML, automation flows A-I)
 2. Complete Source Code (every file in the application)
-

PART 1: SELF-HOSTING GUIDE

Self-Hosting Guide: Dashboard App on Raspberry Pi

Table of Contents

Part 1: Setup

1. [Can I Run This on My Home Assistant Laptop?](#can-i-run-this-on-my-home-assistant-laptop)
2. [Equipment to Buy (Amazon Links)](#equipment-to-buy)
3. [Step 1: Flash the Operating System](#step-1-flash-the-operating-system)
4. [Step 2: First Boot](#step-2-first-boot)
5. [Step 3: Install System Dependencies](#step-3-install-system-dependencies)
6. [Step 4: Set Up the Database](#step-4-set-up-the-database)
7. [Step 5: Create the App Directory](#step-5-create-the-app-directory)
8. [Step 6: Get the Code Out of Replit](#step-6-get-the-code-out-of-replit)
9. [Step 7: Set Up Environment Variables](#step-7-set-up-environment-variables)
10. [Step 8: Setting Up OAuth Credentials](#step-8-setting-up-oauth-credentials-the-hard-part)
11. [Step 9: Initialize the Database](#step-9-initialize-the-database)
12. [Step 10: Build and Test](#step-10-build-and-test)
13. [Step 11: Set Up Auto-Start (systemd Service)](#step-11-set-up-auto-start-systemd-service)
14. [Step 12: Set Up Log Rotation](#step-12-set-up-log-rotation)
15. [Step 13: Update Home Assistant Webhooks](#step-13-update-home-assistant-webhooks)
16. [Step 14: Update Google Apps Script](#step-14-update-google-apps-script)

17. [Step 15: Touchscreen Setup](#step-15-touchscreen-setup)

Part 2: Ongoing Maintenance

18. [Viewing Logs](#viewing-logs)

19. [Restarting / Updating / Backing Up](#restarting-the-app)

20. [Key Differences from Replit](#key-differences-from-replit)

21. [Estimated Migration Effort](#estimated-migration-effort)

Part 3: Home Assistant YAML Reference

22. [REST Commands (configuration.yaml)](#rest-commands-configurationyaml)

23. [HA Automations (automations.yaml)](#ha-automations-automationsyaml)

24. [HA Input Booleans](#ha-input-booleans-configurationyaml)

Part 4: Cat Washroom Study Reading System — Complete Flow

25. [Devices Involved](#devices-involved)

26. [Flow A: Lights Turn ON — Full Prompt Flow](#flow-a-lights-turn-on--full-prompt-flow)

27. [Flow B: Lights Turn OFF — Stop Everything](#flow-b-lights-turn-off--stop-everything)

28. [Flow C: Confirmed Playback Flow (Reading the PDF)](#flow-c-confirmed-playback-flow-reading-the-pdf)

29. [Flow D: Shower Button — Direct Play](#flow-d-shower-button--direct-play-skip-prompt)

30. [Flow E: Volume Knob](#flow-e-volume-knob--adjust-active-speaker-volume)

31. [Flow F: Knob Press — Master STOP](#flow-f-knob-press--master-stop)

32. [Flow G: Toothbrush Auto-Stop](#flow-g-toothbrush-auto-stop)

33. [Flow H: Voice Commands (pause/resume/stop/skip/restart/reset)](#flow-h-voice-commands)

34. [Flow I: Play Urgent PDF](#flow-i-play-urgent-pdf-on-demand)

35. [TTS Fallback Chain](#tts-fallback-chain)

36. [Background Processes](#background-processes-no-webhook-needed)

Can I Run This on My Home Assistant Laptop?

No, not directly. Home Assistant OS is a locked-down, purpose-built Linux distribution — it manages its own filesystem, networking, and containers. You can't install Node.js, PostgreSQL, or run arbitrary apps alongside it. It's designed to only run HA and its add-ons. You'd need to replace HAOS with a regular Linux install to do that, which would mean rebuilding your entire HA setup.

A Raspberry Pi 5 on the same local network is the right approach — fast, cheap, always-on, and won't interfere with your HA installation.

Equipment to Buy

| Item | Model / Spec | Approx. Price | Amazon Link |

|-----|-----|-----|-----|

| **Raspberry Pi 5** | **8 GB RAM** | ~\$80 USD | [amazon.ca/dp/BoCK2FCG1K](https://www.amazon.ca/Raspberry-Pi-8GB-2023-Processor/dp/BoCK2FCG1K) |

| **Power Supply** | Official Raspberry Pi 5 27W USB-C | ~\$12 | [amazon.ca/dp/BoCN1HP2P7](https://www.amazon.ca/dp/BoCN1HP2P7) |

| **microSD Card** | Samsung EVO Select 128 GB (A2-rated) | ~\$13 | [amazon.ca/dp/Bo9B1HMJ9Z](https://www.amazon.ca/dp/Bo9B1HMJ9Z) |

| **Active Cooler** | Official Raspberry Pi 5 Active Cooler | ~\$5 | [amazon.ca/dp/BoCN1GXRKQ](https://www.amazon.ca/dp/BoCN1GXRKQ) |

| **Case** | Official Pi 5 Case (Red/White) | ~\$10 | [amazon.ca/dp/BoCN1HP2RZ](https://www.amazon.ca/dp/BoCN1HP2RZ) |

| **Ethernet Cable** | Cat 6, 3-foot or whatever length you need | ~\$5 | [amazon.ca/dp/BooN2VILDM](https://www.amazon.ca/dp/BooN2VILDM) |

| **USB microSD Reader** | Anker USB 3.0 Card Reader | ~\$8 | [amazon.ca/dp/Boo6T9B6R2](https://www.amazon.ca/dp/Boo6T9B6R2) |

Total: ~\$125–135 USD

Notes:

- The 8 GB Pi 5 is essential — Node.js + PostgreSQL + TTS audio processing need the memory. 4 GB would be tight.
- The official 27W power supply is important — underpowered supplies cause random crashes and SD card corruption.
- The microSD must be A2-rated for the random read/write speed needed by PostgreSQL.
- The active cooler is necessary because TTS processing pushes the CPU under sustained load. A passive heatsink alone won't cut it.
- If your PC already has a microSD slot, skip the USB reader.

Setup Instructions

Step 1: Flash the Operating System

1. On your regular computer, download **Raspberry Pi Imager** from [raspberrypi.com/software](https://www.raspberrypi.com/software/)

2. Insert the microSD card into your computer

3. Open the Imager and choose:

- **Device:** Raspberry Pi 5

- **OS:** Raspberry Pi OS (64-bit, Lite) — no desktop environment needed

4. Click the **gear icon** before flashing and configure:

- **Enable SSH:** Yes

- **Username:** pi

- **Password:** (set a strong password)

- **Wi-Fi:** Enter your network name and password (as backup even if using ethernet)

- **Hostname:** dashboard-server

5. Flash the card and wait for verification to complete

Step 2: First Boot

1. Insert the microSD card into the Pi 5

2. Connect the ethernet cable to your router

3. Plug in the power supply

4. Wait approximately 2 minutes for the first boot to complete

5. From your computer, open a terminal and connect:

```
ssh pi@dashboard-server.local
```

If that doesn't resolve, check your router's admin page for the Pi's IP address and use:

```
ssh pi@<IP_ADDRESS>
```

6. Enter the password you set during flashing

Step 3: Install System Dependencies

Run each block of commands in order:

Update the system

```
sudo apt update && sudo apt upgrade -y
```

Install Node.js 20

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

Install PostgreSQL

```
sudo apt install -y postgresql postgresql-contrib
```

Install build tools (required for some npm packages)

```
sudo apt install -y build-essential git
```

Install espeak-ng (local offline TTS fallback)

```
sudo apt install -y espeak-ng
```

Install Edge TTS (Microsoft TTS fallback)

```
sudo apt install -y python3-pip
```

```
pip3 install edge-tts --break-system-packages
```

Verify installations:

```
node --version          # Should show v20.x.x
```

```
psql --version          # Should show 16.x
```

```
espeak-ng --version     # Should show espeak-ng version
```

```
edge-tts --version      # Should show edge-tts version
```

Step 4: Set Up the Database

Create the database user

```
sudo -u postgres psql -c "CREATE USER dashboard WITH PASSWORD 'CHOOSE_A_STRONG_PASSWORD';"
```

Create the database

```
sudo -u postgres psql -c "CREATE DATABASE dashboard_db OWNER dashboard;"
```

Grant privileges

```
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE dashboard_db TO dashboard;"
```

Replace `CHOOSE_A_STRONG_PASSWORD` with an actual secure password. Write it down — you'll need it for the environment file.

Step 5: Create the App Directory

```
sudo mkdir -p /opt/dashboard  
sudo chown pi:pi /opt/dashboard  
  
cd /opt/dashboard
```

Step 6: Get the Code Out of Replit

Your entire app — every file, every line of code — is yours and can be downloaded at any time. Here are three ways to get it:

Option A — Download as ZIP (easiest, no account needed):

1. In Replit, look at the left sidebar where your files are listed
2. Click the **three dots (...)** at the top of the file panel
3. Click **"Download as ZIP"**

4. A `.zip` file will download to your computer (probably to your Downloads folder)

5. That ZIP contains everything: all the server code, frontend code, database schema, configuration files, this guide — all of it

To transfer the ZIP to your Pi, open a terminal on your regular computer and run:

```
scp ~/Downloads/home-view.zip pi@dashboard-server.local:/opt/dashboard/
```

(If your computer is Windows, you can use WinSCP or drag-and-drop the file using FileZilla instead.)

Then on the Pi (SSH in first):

```
cd /opt/dashboard  
sudo apt install -y unzip      # install unzip if not already there  
  
unzip home-view.zip  
  
npm install
```

Option B — Push to GitHub first (better for ongoing updates):

1. In Replit, click the **Git** icon in the left sidebar (the branch icon)
2. Click "**Connect to GitHub**" and follow the prompts to create a repository
3. Once connected, all your code is on GitHub — you can access it from anywhere, forever

Then on the Pi:

```
cd /opt/dashboard  
git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git .  
  
npm install
```

Future updates become easy: edit on Replit, push to GitHub, then `git pull && npm install && npm run build && sudo systemctl restart dashboard` on the Pi.

Option C — Copy files directly via SCP (no ZIP needed):

If you just want to copy the project folder from your computer after downloading:

```
scp -r ~/Downloads/home-view/* pi@dashboard-server.local:/opt/dashboard/
```

What you're getting:

The download includes absolutely everything that makes the app work:

- `server/routes.ts` — the entire backend (all webhooks, TTS, Spotify, calendar, cat washroom logic)
- `server/storage.ts` — database operations
- `shared/schema.ts` — database table definitions
- `client/src/` — the entire frontend (dashboard, Spotify player, PDF reader, OneNote, settings)
- `package.json` — all the dependency definitions (npm install will download them)
- `Self_Hosting_Guide.md` — this guide
- `HA_Automations_Reference.md` — the full HA automations reference
- Everything else: configuration files, assets, styles

The code is fully self-contained. You don't need Replit to run it — it's a standard Node.js + PostgreSQL app that runs anywhere.

Step 7: Set Up Environment Variables

Create the environment file:

```
nano /opt/dashboard/.env
```

Add the following (fill in your actual values):

Database

```
DATABASE_URL=postgresql://dashboard:CHOOSE_A_STRONG_PASSWORD@localhost:5432/dashboard_db
```

Home Assistant

```
HOME_ASSISTANT_TOKEN=your_long_lived_access_token_from_ha
```

App URL (update to Pi's address)

```
DEPLOYED_APP_URL=http://dashboard-server.local:5000
```

Google Calendar OAuth (from Google Cloud Console)

GOOGLE_CLIENT_ID=your_google_client_id

GOOGLE_CLIENT_SECRET=your_google_client_secret

GOOGLE_REFRESH_TOKEN=your_google_refresh_token

Spotify OAuth (from developer.spotify.com)

SPOTIFY_CLIENT_ID=your_spotify_client_id

SPOTIFY_CLIENT_SECRET=your_spotify_client_secret

SPOTIFY_REFRESH_TOKEN=your_spotify_refresh_token

Microsoft / OneDrive / Outlook (from Azure portal)

MICROSOFT_CLIENT_ID=your_azure_client_id

MICROSOFT_CLIENT_SECRET=your_azure_client_secret

MICROSOFT_REFRESH_TOKEN=your_microsoft_refresh_token

Gmail (from Google Cloud Console – can be same project as Calendar)

GOOGLE_CLIENT_ID=your_google_client_id

```
GMAIL_CLIENT_SECRET=your_gmail_client_secret
```

```
GMAIL_REFRESH_TOKEN=your_gmail_refresh_token
```

Save and exit (Ctrl+X, then Y, then Enter).

Secure the file:

```
chmod 600 /opt/dashboard/.env
```

Step 8: Setting Up OAuth Credentials (The Hard Part)

On Replit, the integration connectors handle OAuth automatically. On the Pi, you need to register apps with each service and get refresh tokens manually.

Google (Calendar + Gmail)

1. Go to console.cloud.google.com
2. Create a new project (or use existing)
3. Enable **Google Calendar API** and **Gmail API**
4. Go to **Credentials**, then Create **OAuth 2.0 Client ID**
5. Application type: **Web application**
6. Add redirect URI: `http://dashboard-server.local:5000/api/google/callback`
7. Copy the Client ID and Client Secret into your `.env`
8. To get a refresh token, you'll need to do a one-time OAuth flow — open this URL in a browser (replace YOUR_CLIENT_ID):

```
https://accounts.google.com/o/oauth2/v2/auth?
client_id=YOUR_CLIENT_ID&redirect_uri=http://dashboard-
server.local:5000/api/google/callback&response_type=code&scope=https://www.googleapis.com/auth/calend
```

9. After authorizing, the callback will receive an authorization code. Exchange it for tokens.

Spotify

1. Go to developer.spotify.com/dashboard
2. Create an app
3. Add redirect URI: `http://dashboard-server.local:5000/api/spotify/callback`
4. Copy Client ID and Client Secret into your `.env`
5. Do the OAuth flow similar to Google to get a refresh token

Microsoft (OneDrive + Outlook)

1. Go to portal.azure.com then Azure Active Directory, then App registrations
2. Register a new application
3. Add redirect URI: `http://dashboard-server.local:5000/api/microsoft/callback`
4. Under **Certificates & secrets**, create a new client secret
5. Under **API permissions**, add: `Files.ReadWrite` , `Mail.Read` , `Calendars.Read`
6. Copy Client ID and Client Secret into your `.env`
7. Do the OAuth flow to get a refresh token

Step 9: Initialize the Database

```
cd /opt/dashboard  
npm run db:push
```

This creates all the tables. If it asks for confirmation, type `yes` .

Step 10: Build and Test

Build the app

```
npm run build
```

Test run (foreground – you'll see logs)

```
node dist/index.js
```

Open a browser on your network and go to:

```
http://dashboard-server.local:5000
```

Verify the dashboard loads. Press Ctrl+C to stop the test.

Step 11: Set Up Auto-Start (systemd Service)

Create the service file:

```
sudo nano /etc/systemd/system/dashboard.service
```

Paste this:

```
[Unit]
```

```
Description=Dashboard App
```

```
After=network.target postgresql.service
```

```
[Service]
```

```
Type=simple
```

```
User=pi
```

```
WorkingDirectory=/opt/dashboard
```

```
EnvironmentFile=/opt/dashboard/.env
```

```
ExecStart=/usr/bin/node dist/index.js
```

```
Restart=always
```

```
RestartSec=10
```

```
StandardOutput=append:/var/log/dashboard.log
```

```
StandardError=append:/var/log/dashboard-error.log
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable dashboard

sudo systemctl start dashboard

sudo systemctl status dashboard
```

The app will now start automatically on every boot and restart if it crashes.

Step 12: Set Up Log Rotation

```
sudo nano /etc/logrotate.d/dashboard
```

Paste:

```
/var/log/dashboard.log /var/log/dashboard-error.log {
    daily

    rotate 7

    compress

    delaycompress

    missingok

    notifempty

    create 0644 pi pi

}
```

Step 13: Update Home Assistant Webhooks

In your HA `configuration.yaml`, update all `rest_command` URLs from:

```
https://home-view--bkh416.replit.app/api/webhook/...
```

To:

```
http://dashboard-server.local:5000/api/webhook/...
```

Or use the Pi's static IP address instead of the hostname for reliability.

See the **Home Assistant YAML Reference** section below for every `rest_command` and `automation` you need.

Step 14: Update Google Apps Script

If you have a Google Apps Script pushing emails to the app, update the webhook URLs there too to point to the Pi's address.

Important: Since the Pi is on your local network, the Google Apps Script (which runs in the cloud) won't be able to reach `dashboard-server.local`. You'll need either:

- A static public IP with port forwarding on your router (port 5000)
- A reverse tunnel service like **Cloudflare Tunnel** (free) to expose the Pi to the internet securely
- Or keep using Replit just for the Gmail webhook endpoint

Step 15: Touchscreen Setup

On your 1920x720 touchscreen, open the browser and navigate to:

```
http://dashboard-server.local:5000
```

Bookmark it or set it as the homepage.

Ongoing Maintenance

Viewing Logs

Live logs

```
sudo journalctl -u dashboard -f
```

Or read the log files

```
tail -100 /var/log/dashboard.log
```

```
tail -100 /var/log/dashboard-error.log
```

Restarting the App

```
sudo systemctl restart dashboard
```

Updating the Code

```
cd /opt/dashboard
```

Transfer new code (scp or git pull)

```
npm install
```

```
npm run build
```

```
sudo systemctl restart dashboard
```

Backing Up the Database

```
pg_dump -U dashboard dashboard_db > /opt/dashboard/backup_$(date +%Y%m%d).sql
```


Restoring a Backup

```
psql -U dashboard dashboard_db < /opt/dashboard/backup_YYYYMMDD.sql
```

Key Differences from Replit

Feature	Replit	Self-Hosted Pi
OAuth Integrations	Managed automatically by Replit connectors	You manage tokens, refresh, and re-auth manually
HTTPS	Automatic	Not included — use Cloudflare Tunnel if needed for external access
Uptime	Replit manages hosting	You manage power, updates, SD card health
External Webhooks	Public URL available immediately	Need port forwarding or tunnel for cloud services to reach the Pi
Database Backups	Replit handles it	You set up cron jobs for backups
Speed	Cloud latency to HA	Local network — faster for all HA communication

Estimated Migration Effort

Task	Time
Hardware setup + OS flash	30 minutes
System dependencies	20 minutes
Code transfer + database init	15 minutes
Google OAuth setup	1-2 hours
Spotify OAuth setup	30 minutes
Microsoft OAuth setup	1-2 hours
HA webhook URL updates	30 minutes
Testing everything	1-2 hours
Total	4-7 hours

The OAuth setup is the bulk of the work. Everything else is straightforward.

Home Assistant YAML Reference

All YAML below goes into your Home Assistant `configuration.yaml` (or split into `automations.yaml` / `rest_commands.yaml` using `!include` if you prefer). Replace `YOUR_APP_URL` with either `https://home-view-bkh416.replit.app` (Replit) or `http://dashboard-server.local:5000` (self-hosted Pi).

REST Commands (`configuration.yaml`)

These define the HTTP calls HA can make to your app. Every webhook endpoint needs one.

```
rest_command:

  # Cat Washroom Lights – triggers study reading prompt or stop

cat_lights_webhook:

  url: "YOUR_APP_URL/api/webhook/cat-lights"

  method: POST

  headers:

    Content-Type: "application/json"

  payload: '{"state": "{ { states(''light.cat_lights'') } }"}'

  # Cat Washroom Lights – confirmation ("yes, play the reading")

cat_lights_confirm_webhook:

  url: "YOUR_APP_URL/api/webhook/cat-lights-confirm"

  method: POST

  headers:

    Content-Type: "application/json"

  # Cat Washroom Shower Button – direct play (skips the prompt)

cat_shower_button_webhook:

  url: "YOUR_APP_URL/api/webhook/cat-shower-button"
```

```
method: POST
```

```
headers:
```

```
Content-Type: "application/json"
```

```
# Cat Washroom Stop — stops playback and saves progress
```

```
cat_wash_stop_webhook:
```

```
url: "YOUR_APP_URL/api/webhook/cat-wash-stop"
```

```
method: POST
```

```
headers:
```

```
Content-Type: "application/json"
```

```
payload: '{"trigger":"{{ trigger }}"}'
```

```
# Cat Washroom Volume Knob — adjusts active speaker volume
```

```
cat_volume_webhook:
```

```
url: "YOUR_APP_URL/api/webhook/cat-volume"
```

```
method: POST
```

```
headers:
```

```
Content-Type: "application/json"
```

```
payload: '{"direction":"{{ direction }}","speed":"{{ speed }}"}'
```

```
# Cat Washroom Knob Press — master STOP button
```

```
cat_knob_press_webhook:
```

```
url: "YOUR_APP_URL/api/webhook/cat-knob-press"
```

method: POST

headers:

Content-Type: "application/json"

Voice Commands — pause, resume, stop, skip, restart, reset

voice_command_webhook:

url: "YOUR_APP_URL/api/webhook/voice-command"

method: POST

headers:

Content-Type: "application/json"

payload: '{"command":"{{ command }}"}'

Kitchen Volume Knob — adjusts Kitchen Echo Studio volume

kitchen_volume_webhook:

url: "YOUR_APP_URL/api/webhook/kitchen-volume"

method: POST

headers:

Content-Type: "application/json"

payload: '{"direction":"{{ direction }}","speed":"{{ speed }}"}'

Play Urgent PDF — on-demand play most important unlistened reading

play_urgent_pdf_webhook:

url: "YOUR_APP_URL/api/webhook/play-urgent-pdf"

```
method: POST
```

```
headers:
```

```
Content-Type: "application/json"
```

```
x-webhook-secret: "YOUR_SITE_PASSWORD"
```

```
payload: '{"entity_id":"media_player.bathroom_speaker"}'
```

HA Automations (automations.yaml)

Automation 1: Cat Washroom Lights Changed

Fires whenever the cat washroom lights turn on or off. The app checks the actual state and either starts the study prompt flow (on) or stops everything (off).

```
automation:
- alias: "Cat Washroom Lights Changed"

  trigger:

    - platform: state

      entity_id: light.cat_lights

  action:

    - service: rest_command.cat_lights_webhook
```

Automation 2: Cat Washroom Lights Confirmation

Fires when you confirm (button press, voice, or HA toggle) that you want to hear the reading. This resolves the 23-second wait from Automation 1.

```
automation:
  - alias: "Cat Washroom Reading Confirmed"

  trigger:

    - platform: state

      entity_id: input_boolean.module_reading_confirmed

      to: "on"

  action:

    - service: rest_command.cat_lights_confirm_webhook
```

Automation 3: Cat Washroom Shower Button

Fires when the shower button entity changes. Skips the prompt and goes straight into playback.

```
automation:
  - alias: "Cat Washroom Shower Button"

  trigger:

    - platform: state

      entity_id: switch.cat_shower_button # or whatever your shower button entity is

  action:

    - service: rest_command.cat_shower_button_webhook
```

Automation 4: Cat Washroom Volume Knob (Rotate)

Fires when the rotary encoder sends a rotation event. Your ESPHome or Zigbee config should set the `direction` variable to "up" or "down" and `speed` to "normal" or "fast".

automation:

- alias: "Cat Washroom Volume Up"

trigger:

- platform: state

entity_id: sensor.cat_volume_knob_rotation # your encoder entity

action:

- service: rest_command.cat_volume_webhook

data:

direction: "{{ 'up' if trigger.to_state.state | float > trigger.from_state.state | float
else 'down' }}"

speed: "normal"

- alias: "Cat Washroom Volume Fast"

trigger:

- platform: state

entity_id: sensor.cat_volume_knob_fast_rotation # fast rotation entity

action:

- service: rest_command.cat_volume_webhook

data:

direction: "{{ 'up' if trigger.to_state.state | float > trigger.from_state.state | float
else 'down' }}"

speed: "fast"

Automation 5: Cat Washroom Knob Press (STOP)

Fires when you press the physical knob button. Stops everything.

```
automation:
  - alias: "Cat Washroom Knob Press STOP"

  trigger:

    - platform: state

      entity_id: binary_sensor.cat_volume_knob_press # your knob press entity

      to: "on"

  action:

    - service: rest_command.cat_knob_press_webhook
```

Automation 6: Kitchen Volume Knob

Same pattern as the cat washroom volume knob, but for the kitchen.

automation:

- alias: "Kitchen Volume Up"

trigger:

- platform: state

entity_id: sensor.kitchen_volume_knob_rotation

action:

- service: rest_command.kitchen_volume_webhook

data:

direction: "{{ 'up' if trigger.to_state.state | float > trigger.from_state.state | float
else 'down' }}"

speed: "normal"

HA Input Booleans (configuration.yaml)

These are required for the confirmation flow. The app sets them during the lights prompt and polls them as a backup confirmation method.

```
input_boolean:

  module_reading_pending:

    name: Module Reading Pending

    icon: mdi:book-open-page-variant


module_reading_confirmed:

  name: Module Reading Confirmed

  icon: mdi:check-circle
```

Cat Washroom Study Reading System — Complete Flow

Overview

The cat washroom has a study reading system that plays your school readings aloud on the Nest speaker while displaying the text on a Fire tablet and Samsung TV. It is triggered by turning on the cat washroom lights or pressing the shower button, and can be stopped in multiple ways.

Devices Involved

Device	HA Entity	Role
----- ----- -----		
Cat Washroom Lights	light.cat_lights	Trigger (on/off)
Google Nest Speaker	media_player.bathroom_speaker	Main audio playback (TTS reading)
HA Voice ESPHome Device	media_player.home_assistant_voice_097c38_media_player	Voice prompts ("Would you like to play...?")
Fire Tablet (Cat WR)	media_player.tablet_cat	Displays PDF reader page with synced text
Samsung TV (Cat WR)	media_player.tv_cat_wr	Displays PDF reader follow-along page
Fire Stick (Cat WR TV)	media_player.fire_tv_172_24_0_88	Drives Samsung TV display via HDMI
Echo Cat Left	media_player.echo_cat_left_am	CHUM FM playback (Echo speaker group)
Echo Cat Right	media_player.echo_cat_right_am	CHUM FM playback (Echo speaker group)

Echo Cat Middle	`media_player.echo_cat_washroom_middle`	CHUM FM playback (Echo speaker group)
Cat WR Media Group	`media_player.cat_washroom_media_group`	Alexa multi-room group for CHUM FM
HA Cloud TTS	`tts.home_assistant_cloud`	Nabu Casa cloud TTS (last-resort prompt voice)
Oral-B Toothbrush	`sensor.toothbrush_bryn_toothbrush_state`	Auto-stop trigger (polled, not webhook)

FLOW A: Lights Turn ON — Full Prompt Flow

Trigger: `light.cat_lights` state changes (HA automation calls `rest_command.cat_lights_webhook`)

Endpoint: `POST /api/webhook/cat-lights`

Step-by-step:

1. Guards (any of these abort the flow):

- Server startup cooldown (skips if server just restarted within cooldown window)
- Already playing a reading (skips if `catWashPlaybackActive` is true)
- Another prompt is already pending (skips if `catLightsPromptPending` is true)

2. Query the actual light state from HA:

- `GET /api/states/light.cat_lights`
- If state is "off" — go to **FLOW B** (Lights Turn OFF) instead
- If state is not "on" — ignore the webhook

3. Immediate acknowledgment (plays right away while the app looks up files):

- Set volume on HA Voice device to 0.64
- Set `input_boolean.module_reading_confirmed` to OFF
- Set `input_boolean.module_reading_pending` to ON
- Play via HA Cloud TTS on the HA Voice device:

> **"One moment, checking your readings."**

4. Look up the next unlistened file:

- Get active semester settings from the database
- Calculate the current week number (accounting for reading week)
- Search database for the next unlistened file for this week, prioritized:
 1. Module files first (sorted by course priority)
 2. Reading files second (sorted by course priority)
- If no cached files found, sync from OneDrive first, then search again

5. Light re-check:

- Query `light.cat_lights` state again
- If the light turned OFF while the app was looking up files — abort silently

6. If NO unlistened files found:

- Play CHUM FM 104.5 on the Echo speaker group:
- `media_player/play_media on media_player.cat_washroom_media_group`
- `media_content_type: "custom", media_content_id: "play 104.5 chum fm"`
- Done. No prompt.

7. If an unlistened file IS found — play the voice prompt:

- Stop any leftover media on all cat washroom speakers first
- Set Nest speaker volume to 0.64
- Generate the prompt message. Example:
> **"Would you like to play week 8, C.P.P.A. 1 22 module?"**
- Try to play via HA Cloud TTS on the HA Voice device
- If that fails, fall back to Edge TTS audio file played on the HA Voice device
- If ALL TTS methods fail, give up and play CHUM FM instead

8. Wait 2 seconds for the prompt to finish speaking

9. Light re-check #2:

- Query `light.cat_lights` again
- If OFF — abort, reset booleans, done

10. Wait for confirmation (up to 23 seconds):

- Primary: wait for `POST /api/webhook/cat-lights-confirm` (resolves the promise immediately)
- Backup: poll `input_boolean.module_reading_confirmed` every 10 seconds
- Timeout after 23 seconds

11. Reset HA booleans regardless of outcome:

- `input_boolean.module_reading_pending OFF`
- `input_boolean.module_reading_confirmed OFF`

12a. If NOT confirmed (23-second timeout):

- Play CHUM FM 104.5 on Echo speaker group
- Done.

12b. If CONFIRMED:

- Light re-check #3 (abort if OFF)
 - Play confirmation voice:
> **"Okay, I will now play week 8, C.P.P.A. 1 22 module."**
 - Start the full playback flow (**FLOW C** below)
-

FLOW B: Lights Turn OFF — Stop Everything

Trigger: `light.cat_lights` state changes to OFF

Endpoint: `POST /api/webhook/cat-lights` (same endpoint, app checks state)

Step-by-step:

1. If a reading is actively playing:

- Save current chunk progress to database
- Stop Nest speaker: `media_player/media_stop on media_player.bathroom_speaker`
- Turn off Fire Stick: `media_player/turn_off on media_player.fire_tv_172_24_0_88`
- Turn off Samsung TV: `media_player/turn_off on media_player.tv_cat_wr`
- Play goodbye TTS on Nest speaker:
> **"Stopping. [File Name]. The file position has been saved. See you next time Bryn."**
- Send `stop_playback` command to tablet and TV displays
- Stop toothbrush polling
- Clear playback session from database

2. If a TTS session is active:

- Stop the TTS session

3. Stop all cat washroom speakers (always, even if nothing was playing):

- `media_player/media_stop on media_player.bathroom_speaker`
- `media_player/media_stop on all 3 Echo speakers ( echo_cat_left_am , echo_cat_right_am , echo_cat_washroom_middle )`
- `media_player/media_stop on media_player.cat_washroom_media_group`

4. Reset state flags:

- `catLightsPromptPending = false`
 - `catWashPlaybackTrigger = null`
-

FLOW C: Confirmed Playback Flow (Reading the PDF)

Triggered by: Flow A confirmation, shower button (Flow D), voice commands (resume/restart/reset/skip), or play-urgent-pdf webhook.

Step-by-step:

1. Calculate resume point:

- Look up `lastChunkIndex` from the database for this file
- Resume from `max(0, savedChunk - 1)` — goes back 1 chunk for context overlap

2. Build URLs:

- Tablet URL: `/pdf-reader/{fileId}?catWashFollow=true&resumeChunk={N}&voice=echo&fullscreen=true`
- TV follow URL: `/pdf-reader/{fileId}?catWashFollow=true&followOnly=true&...`

3. PARALLEL SETUP (all happen simultaneously):

Tablet Setup (`media_player.tablet_cat`):

- ADB: `input keyevent KEYCODE_WAKEUP` — wake the tablet
- ADB: `settings put system screen_brightness 255` — max brightness
- Wait 1.5 seconds
- ADB: `am start --activity-clear-task -a android.intent.action.VIEW -d "{readerUrl}" com.amazon.cloud9` — open PDF reader in Silk browser
- Wait 1 second
- ADB: `settings put global policy_control immersive.full=com.amazon.cloud9` — immersive mode (hide status/nav bars)
- ADB: `input keyevent KEYCODE_F11` — fullscreen

TV Setup:

- `media_player/turn_on` on `media_player.fire_tv_172_24_0_88` — wake Fire Stick
- `media_player/turn_on` on `media_player.tv_cat_wr` — turn on Samsung TV
- Wait 3 seconds (let TV boot)
- `media_player/select_source` on `media_player.tv_cat_wr`, source: "HDMI1" — switch to Fire Stick input
- ADB on Fire Stick: wake, kill existing Silk browser, open redirect URL, set immersive mode, press center button to dismiss dialogs

Pre-generate first audio chunk:

- Check if pre-prepared audio exists (from background 30-min prep job)
- If yes, use the cached .mp3 file
- If no, generate via Edge TTS (voice: `en-US-AndrewMultilingualNeural`)

Confirmation TTS (plays on Nest speaker):

> "Okay, I will now play [file description]."

- Wait for estimated duration (word count / 140 wpm)

4. Set playback volume:

- `media_player/volume_set` on HA Voice device @ 0.75
- `media_player/volume_set` on Nest speaker @ 0.75

5. Start chunk-by-chunk reading loop:

For each chunk (starting from resume point):

- Check session is still valid (not cancelled, not stale)

- Generate or use pre-prepared TTS audio for this chunk
- Play on Nest speaker:
 - `media_player/media_stop on media_player.bathroom_speaker`
 - `media_player/volume_set @ 0.75`
 - `media_player/play_media` with the audio URL
- Wait for estimated chunk duration (word count / 175 wpm + 1 second buffer)
- Poll Nest speaker state to confirm playback finished
- Update database: `lastChunkIndex` , `checkedChunks`
- Send progress to tablet and TV via polling endpoint (they update the highlighted text)
- Pre-generate NEXT chunk audio in background (pipeline)

6. When ALL chunks are complete:

- Mark file as "listened" in database
- Play goodbye TTS:
 - > **"All done with [file name]. Great work Bryn!"**
- Look for the next unlistened file
- If another file exists: start playing it automatically (back to step 1)
- If no more files: stop playback, turn off Fire Stick and Samsung TV

7. Start toothbrush polling (runs alongside playback):

- Poll `sensor.toothbrush_bryn_toothbrush_state` every 3 seconds
- If state changes to "running" or "brushing" , auto-stop playback (see Flow F)

FLOW D: Shower Button — Direct Play (Skip Prompt)

Trigger: Shower button entity changes in HA

Endpoint: `POST /api/webhook/cat-shower-button`

Step-by-step:

1. Guards:

- Server startup cooldown
- Skip if playback already active (assumes you're toggling the shower/fan)
- Skip if a lights prompt is already pending

2. Look up semester and find next file (same as Flow A, steps 4-6)

3. If no unlistened files:

- Play CHUM FM 104.5 on Echo speaker group
- Done.

4. If file found — go straight to playback (no prompt, no 23-second wait):

- Play confirmation:

> "Okay, I will now play [file description]."

- Start **Flow C** (confirmed playback) immediately

FLOW E: Volume Knob — Adjust Active Speaker Volume

Trigger: Rotary encoder rotation in HA

Endpoint: POST /api/webhook/cat-volume

Body: { "direction": "up" | "down", "speed": "normal" | "fast" }

Step-by-step:

1. Query which speakers are currently active:

- Check state of `media_player.bathroom_speaker` (Nest)
- Check state of `media_player.cat_washroom_media_group` (Echo group)
- Active = state is "playing", "paused", or "buffering"
- If none active, default to Nest speaker

2. Calculate new volume:

- Normal speed: current volume +/- 0.05 (5% per click)
- Fast speed: current volume +/- 0.15 (15% per click)
- Clamped between 0.0 and 1.0

3. Set volume on each active speaker:

- `media_player/volume_set` on each active entity
-

FLOW F: Knob Press — Master STOP

Trigger: Physical knob button press in HA

Endpoint: POST /api/webhook/cat-knob-press

Step-by-step:

1. If a reading is actively playing:

- Internally calls POST /api/webhook/cat-wash-stop with { "trigger": "knob_press", "keepOpen": true }
- This triggers the full stop flow: save progress, stop Nest, turn off TV/Fire Stick, play goodbye
- Goodbye TTS:

> **"Stopping. [File Name]. The file position has been saved. See you next time Bryn."**

2. If nothing is playing:

- Stop all cat washroom speakers (kills CHUM FM, any leftover media):
 - `media_player/media_stop` on Nest, all 3 Echos, and the media group
-

FLOW G: Toothbrush Auto-Stop

Trigger: App polls `sensor.toothbrush_bryn_toothbrush_state` every 3 seconds (NOT a webhook)

No HA automation needed — the app does this internally whenever playback starts.

Step-by-step:

1. **Polling starts** when any playback begins (Flow C)

2. Each poll:

- GET `/api/states/sensor.toothbrush_bryn_toothbrush_state` from HA
- If state is "running" or "brushing" :
- Stop playback via internal `cat-wash-stop` logic
- Save progress to database
- Stop Nest speaker
- Turn off Fire Stick and Samsung TV
- Play goodbye:

> **"Stopping. Your reading position has been saved. See you next time Bryn."**

3. **Polling stops** when playback ends (any stop mechanism)

FLOW H: Voice Commands

Trigger: HA voice assistant or custom intent

Endpoint: POST `/api/webhook/voice-command`

Body: { "command": "<command>" }

PAUSE

```
{ "command": "pause" }
```

- If nothing playing: **"Nothing is playing right now."**
- Save chunk progress, stop Nest speaker, stop word advancement on tablet
- Start 10-minute auto-stop timer (if no resume within 10 min, everything shuts off):

- Auto-stop TTS: **"Pause timed out. Playback has been stopped. Your progress has been saved."**

- TTS response:

> **"Paused. Say resume to continue, or I'll stop in 10 minutes."**

RESUME

```
{ "command": "resume" }
```

- If nothing paused: **"Nothing is paused right now."**

- Clear the 10-minute auto-stop timer

- Stop Echo speakers (clear any CHUM FM that might have started)

- TTS response:

> **"Resuming [file description]."**

- Restart playback from saved chunk position (**Flow C**)

STOP

```
{ "command": "stop" }
```

- If nothing playing or paused: **"Nothing is playing."**

- If actively playing: save progress, stop Nest, turn off TV/Fire Stick

- TTS response:

> **"Stopped. Your progress has been saved. See you next time Bryn."**

SKIP (next file)

```
{ "command": "skip" }
```

- If nothing playing: **"Nothing is playing to skip."**

- Mark current file as "listened" (complete) in database

- Stop current playback

- Find next unlistened file

- If no more files: **"Skipped. No more readings for this week. Great work Bryn!"** (turn off TV/Fire Stick)

- If next file found: **"Skipped. Now playing [file description]."** (start Flow C with new file)

RESTART (go back one chunk)

```
{ "command": "restart" }
```

- If nothing playing: **"Nothing is playing to restart."**
- Go back 1 chunk from current position (minimum chunk 0)
- Stop current playback, save new position
- TTS response:
- > **"Going back. Restarting from an earlier section."**
- Restart playback from earlier chunk (**Flow C**)

RESET (start from beginning)

```
{ "command": "reset" }
```

- If nothing playing: **"Nothing is playing to reset."**
 - Reset file progress in database: `lastChunkIndex = 0 , checkedChunks = []`
 - Stop current playback
 - TTS response:
 - > **"Resetting [file description]. Starting from the beginning."**
 - Start playback from chunk 0 (**Flow C**)
-

FLOW I: Play Urgent PDF (On-Demand)

Trigger: HA script, button, or automation

Endpoint: `POST /api/webhook/play-urgent-pdf`

Auth: Requires `x-webhook-secret` header matching your site password

Step-by-step:

1. Authenticate via header

2. Find all unlistened files, ordered by priority:

1. Module files first (sorted by course)
2. Reading files second (sorted by course)

3. If no files:

- TTS on target speaker:
- > **"All week [N] readings are complete. Great job!"**

4. If file found:

- Start **Flow C** (confirmed playback) immediately
-

TTS Fallback Chain

The app uses a 3-level fallback chain for generating speech audio:

| Priority | Engine | Voice | When It's Used |

|-----|-----|-----|-----|

| 1 | OpenAI TTS | alloy | Primary (rate-limited) |

| 2 | Edge TTS (Microsoft) | en-US-AndrewMultilingualNeural | When OpenAI is rate-limited or fails |

| 3 | espeak-ng (local) | Default English | When Edge TTS has 5+ consecutive failures |

For voice prompts (the "Would you like to play?" question), the app uses HA Cloud TTS (Nabu Casa) as the primary method since it's the fastest (no file generation needed), with Edge TTS file playback as the fallback.

Known Issues & Learnings (Nest Speaker)

These are hard-won lessons from debugging the cat washroom reading system. Keep them in mind when modifying the speaker playback code.

1. Nest Speaker State Is Unreliable — Always Trust the Service Call

The Google Nest speaker frequently reports its state as "unknown" even when it IS actively playing audio. The `media_player` entity in HA does not reliably transition to "playing" or "buffering" for Cast-based playback.

Rule: If the `media_player/play_media` service call returns 200 OK from Home Assistant, treat it as SUCCESS. Do NOT use the speaker's reported state (`idle` , `unknown` , `unavailable`) to determine whether playback actually started. The only failure case is when the HA service call itself throws an error (network failure, HA offline, etc.).

If you add a state check after `play_media` , treat "unknown" the same as "playing" — return success. Only retry on "idle" or "off" , and even then, after max retries, still return success since the command was accepted by HA.

2. The Nest CAN Play Audio from the Deployed App URL

The Nest speaker successfully plays MP3 files served from `https://home-view--bkh416.replit.app/api/tts-audio/...` . The `/api/tts-audio/` endpoint is explicitly excluded from authentication in `server/index.ts` (line ~190), so no cookies or tokens are needed.

Do NOT add an HA local media upload step — it adds complexity without benefit. The direct URL approach works. If playback appears broken, the issue is almost certainly the state check logic (see point 1), not the URL accessibility.

3. Circuit Breaker Must Not Fire on State-Check Failures

The chunk playback loop has a circuit breaker that triggers after 3 consecutive "failures." If `playOnNestSpeaker` incorrectly returns `{success: false}` due to an "unknown" state, the circuit breaker fires and prompts the user to switch to Echo speakers — even though the Nest was playing fine.

Rule: `playOnNestSpeaker` should only return `{success: false}` when the HA service call itself fails (HTTP error, timeout, HA offline). State check results should never cause a `{success: false}` return.

4. Volume Levels

Context Volume
----- -----
Voice prompts ("Would you like to play?") 0.35
HA Voice confirmation prompt 0.45
Nest pre-confirm / playback 0.75
Fallback / cat-lights volume 0.35

5. Confirm TTS Plays on the Nest, Not HA Voice

The confirmation message ("Okay, I will now play your module...") should play on the Nest speaker using a generated OpenAI/Edge TTS audio file, not via HA Cloud TTS on the HA Voice speaker. The fallback to HA Voice should only happen if the Nest play_media service call actually fails (HTTP error).

6. Stale Playback Sessions

The webhook handler checks for stale playback at the start. A session is considered stale if:

- It has been running for 3+ minutes AND is still at chunk 0 (stuck at start)
- It has been running for 10+ minutes regardless of chunk position

Stale sessions are cleared so the next light trigger can start fresh.

Background Processes (No Webhook Needed)

These run automatically inside the app — no HA automation required:

Process Interval What It Does
----- ----- -----
Audio Preparation Every 30 minutes Pre-generates TTS audio for upcoming files. Also retries files with failed chunks.
Semester Auto-Activation Every 6 hours + startup Checks all semesters and activates the one whose date range includes today.
Ticker Push to HA Every 5 minutes Fetches weather, news, pollen, course announcements and pushes to <code>sensor.dashboard_ticker</code> , <code>sensor.dashboard_weather</code> , <code>sensor.dashboard_news</code> in HA.
Toothbrush Polling Every 3 seconds (only during playback) Polls toothbrush sensor state. Auto-stops reading if brushing detected.
Alexa Reminder Scheduler Every 60 seconds Checks for due task reminders and sends Echo voice announcements + iPhone push notifications.
Playback Session Persistence On every chunk change Saves current playback state to <code>app_state</code> table so playback can resume after server restart.

Complete Integration & OAuth Setup Guide — Self-Hosting Edition

This guide covers **every external service** your app connects to, what each one does, how to set it up from scratch on a Raspberry Pi, and — critically — which parts of the code need to change because they currently rely on Replit's built-in connectors.

Table of Contents

1. [Overview: What Connects to What](#overview)
 2. [Tools You'll Need](#tools-youll-need)
 3. [CRITICAL: Replit Connector Rewrites](#critical-replit-connector-rewrites)
 4. [Integration 1: Home Assistant](#integration-1-home-assistant)
 5. [Integration 2: Google Calendar](#integration-2-google-calendar)
 6. [Integration 3: Gmail](#integration-3-gmail)
 7. [Integration 4: Spotify](#integration-4-spotify)
 8. [Integration 5: Microsoft OneDrive](#integration-5-microsoft-onedrive)
 9. [Integration 6: Microsoft Outlook Calendar](#integration-6-microsoft-outlook-calendar)
 10. [Integration 7: OpenAI (TTS)](#integration-7-openai-tts)
 11. [Integration 8: Resend (Email Sending)](#integration-8-resend-email-sending)
 12. [Integration 9: Second Google Account (Partner Shifts)](#integration-9-second-google-account)
 13. [Integration 10: Third Google Account (CRCU)](#integration-10-third-google-account)
 14. [Integration 11: Object Storage](#integration-11-object-storage)
 15. [App Security: Site Password & Sessions](#app-security)
 16. [Complete .env Template](#complete-env-template)
 17. [Testing Each Integration](#testing-each-integration)
 18. [ChatGPT Prompts for Integration Help](#chatgpt-prompts)
-

Overview

Your app talks to **10 external services**. Here's what each one does:

| # | Service | What It Does in Your App | Auth Method |

|---|-----|-----|-----|

- | 1 | Home Assistant | Smart home control, speaker playback, sensor data, webhooks | Long-lived access token |
- | 2 | Google Calendar | Shows academic calendar events on dashboard | OAuth 2.0 (currently Replit connector) |
- | 3 | Gmail | Reads D2L announcement emails, sends emails | OAuth 2.0 (currently Replit connector) |
- | 4 | Spotify | Music player on dashboard (play, pause, playlists) | OAuth 2.0 (already uses direct tokens) |
- | 5 | Microsoft OneDrive | Syncs PDF course files for TTS reading | OAuth 2.0 (currently Replit connector) |
- | 6 | Microsoft Outlook | Shows Outlook calendar events on dashboard | OAuth 2.0 (currently Replit connector) |
- | 7 | OpenAI | Generates TTS audio from PDF text | API key (currently Replit AI integration) |
- | 8 | Resend | Sends reminder emails (task due dates, daily digest) | API key |
- | 9 | Second Google Account | Partner's work shift calendar | OAuth 2.0 (direct, already works) |
- | 10 | Third Google Account | CRCU partner shifts | OAuth 2.0 (direct, already works) |

Plus one infrastructure service:

- | 11 | Object Storage | Stores uploaded PDFs and TTS audio files | Replit-managed (needs replacement) |

Tools You'll Need

Before you start, install these on your regular computer (not the Pi):

Required Tools

| Tool | What It's For | Where to Get It |

|-----|-----|-----|

- | **Web Browser** (Chrome/Edge) | Setting up OAuth apps, approving permissions | Already have it |
- | **Terminal / SSH** | Connecting to the Pi, running commands | Built into Mac/Linux. Windows: use PowerShell or install PuTTY |
- | **Text Editor** | Editing .env files and code | VS Code (free, code.visualstudio.com) or Notepad++ |

Helpful But Optional

| Tool | What It's For | Where to Get It |

|-----|-----|-----|

- | **Postman** | Testing API calls and OAuth flows manually | Free at postman.com/downloads |
- | **jq** | Pretty-printing JSON in the terminal | `sudo apt install jq` on the Pi |
- | **curl** | Making test HTTP requests from command line | Already installed on Pi |

Accounts You'll Need

| Account | URL | What For |

|-----|-----|-----|

Google Cloud Console	console.cloud.google.com	Google Calendar + Gmail APIs
Spotify Developer	developer.spotify.com	Spotify music control
Azure Portal	portal.azure.com	OneDrive + Outlook
OpenAI Platform	platform.openai.com	TTS audio generation
Resend	resend.com	Sending reminder emails

CRITICAL: Replit Connector Rewrites

This is the most important section. Four of your integrations currently work through Replit's connector system, which handles OAuth automatically. On a Pi, this system doesn't exist. You need to rewrite the auth code in four files.

What Needs to Change

The following files each have a `getAccessToken()` function that calls Replit's connector API. On a Pi, this won't work — you need to replace it with direct OAuth using your own client ID, client secret, and refresh token.

| File | Connector Used | What To Replace |

|-----|-----|-----|

| server/googleCalendar.ts | google-calendar | getAccessToken() function (lines 1-30) |

| server/gmail.ts | google-mail | getAccessToken() function (lines 1-35) |

| server/onedrive.ts | onedrive | getAccessToken() function (lines 1-35) |

| server/outlookCalendar.ts | outlook | getOutlookAccessToken() function (lines 1-40) |

The Pattern to Replace

Each file currently has code like this (DON'T use this on the Pi):


```
// THIS IS THE REPLIT CONNECTOR CODE — DOES NOT WORK ON PI

const hostname = process.env.REPLIT_CONNECTORS_HOSTNAME;

const xReplitToken = process.env.REPL_IDENTITY

? 'repl ' + process.env.REPL_IDENTITY

: process.env.WEB_REPL_RENEWAL

? 'depl ' + process.env.WEB_REPL_RENEWAL

: null;

connectionSettings = await fetch(

  'https://' + hostname + '/api/v2/connection?include_secrets=true&connector_names=google-
calendar',

  { headers: { 'X_REPLIT_TOKEN': xReplitToken } }

).then(res => res.json()).then(data => data.items?.[0]);
```

The Replacement Pattern

Replace each `getAccessToken()` with this pattern (customize for each service):

```
// DIRECT OAUTH — WORKS ON PI

let cachedAccessToken: string | null = null;

let tokenExpiresAt: number = 0;

async function getAccessToken(): Promise<string> {

  // Return cached token if still valid (with 5-minute buffer)

  if (cachedAccessToken && Date.now() < tokenExpiresAt - 5 * 60 * 1000) {

    return cachedAccessToken;
```

```

}

const clientId = process.env.GOOGLE_CLIENT_ID;          // Change per service

const clientSecret = process.env.GOOGLE_CLIENT_SECRET; // Change per service

const refreshToken = process.env.GOOGLE_REFRESH_TOKEN; // Change per service

if (!clientId || !clientSecret || !refreshToken) {

    throw new Error('Google OAuth credentials not configured');

}

const response = await fetch('https://oauth2.googleapis.com/token', {

    method: 'POST',

    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },

    body: new URLSearchParams({

        client_id: clientId,

        client_secret: clientSecret,

        refresh_token: refreshToken,

        grant_type: 'refresh_token',

    }),

});

if (!response.ok) {

    const error = await response.text();

    throw new Error( Failed to refresh Google token: ${error} );

```

```
}

const data = await response.json();

cachedAccessToken = data.access_token;

tokenExpiresAt = Date.now() + (data.expires_in * 1000);

return cachedAccessToken!;

}
```

The same pattern works for Microsoft — just change the token URL and env var names.

ChatGPT Prompt for Rewriting Connector Code

If you want ChatGPT to do the rewrite for you, paste this:

[Paste the intro statement from the Troubleshooting Guide]

I need to rewrite 4 files that currently use Replit's connector system for OAuth. On my self-hosted Pi, Replit connectors don't exist. I need to replace the `getAccessToken()` function in each file to use direct OAuth 2.0 refresh token flow instead.

Here is the current code for [paste the file name]:

[paste the first 40 lines of the file]

Please rewrite the `getAccessToken()` function to use direct OAuth with these environment variables:

- For Google: `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_REFRESH_TOKEN`
- For Microsoft: `MICROSOFT_CLIENT_ID`, `MICROSOFT_CLIENT_SECRET`, `MICROSOFT_REFRESH_TOKEN`

The rest of the file should stay exactly the same – only the auth function needs to change. The function should:

1. Cache the access token in memory
2. Refresh it automatically when it expires (with a 5-minute buffer)
3. Use the standard OAuth 2.0 `refresh_token` grant flow
4. Throw a clear error if environment variables are missing

Integration 1: Home Assistant

Difficulty: Easy

Already works on Pi: Yes (no code changes needed)

What You Need

- Your Home Assistant URL (either Nabu Casa cloud URL or local IP)
- A long-lived access token

How to Get the Token

1. Open Home Assistant in your browser

2. Click your name/profile in the bottom-left corner
3. Scroll down to "Long-Lived Access Tokens"
4. Click "Create Token"
5. Give it a name like "Dashboard App"
6. Copy the token immediately — you can't see it again

Environment Variables

```
HOME_ASSISTANT_TOKEN=eyJ0eX...your_very_long_token_here  
HOME_ASSISTANT_URL_OVERRIDE=https://your-nabu-casa-url.ui.nabu.casa
```

If your Pi is on the same network as HA, you can also use the local URL:

```
HOME_ASSISTANT_URL_OVERRIDE=http://192.168.1.XXX:8123
```

(Local is faster but only works when on the same network.)

Test It

```
curl -s -H "Authorization: Bearer YOUR_TOKEN" \  
https://YOUR_HA_URL/api/ | jq .
```

Should show: {"message": "API running."}

Integration 2: Google Calendar

Difficulty: Medium-Hard

Requires code change: YES (Replit connector rewrite)

Step 1: Create a Google Cloud Project

1. Go to **console.cloud.google.com**
2. Click the project dropdown at the top, then "New Project"
3. Name it something like "Dashboard App"
4. Click "Create"

Step 2: Enable the Calendar API

1. In your new project, go to **APIs & Services** in the left sidebar, then **Library**
2. Search for "Google Calendar API"
3. Click it, then click **Enable**

Step 3: Set Up OAuth Consent Screen

1. Go to **APIs & Services**, then **OAuth consent screen**
2. Choose **External** (unless you have a Google Workspace org, then choose Internal)
3. Fill in:
 - App name: "Dashboard"
 - User support email: your email
 - Developer email: your email
4. Click "Save and Continue"
5. On the "Scopes" page, click "Add or Remove Scopes"
6. Add: `https://www.googleapis.com/auth/calendar.readonly`
7. Click "Save and Continue"
8. On "Test users", click "Add Users"
9. Add your Gmail address
10. Click "Save and Continue"

Step 4: Create OAuth Credentials

1. Go to **APIs & Services**, then **Credentials**
2. Click **Create Credentials**, then **OAuth 2.0 Client ID**
3. Application type: **Web application**
4. Name: "Dashboard Calendar"
5. Under "Authorized redirect URIs", add:

`http://dashboard-server.local:5000/api/google/callback`

(also add your Pi's IP version: `http://192.168.1.XXX:5000/api/google/callback`)

6. Click "Create"
7. **Copy the Client ID and Client Secret** — save them somewhere safe

Step 5: Get a Refresh Token (The Tricky Part)

You need to do a one-time browser authorization to get a refresh token. Here's how:

5a. Open this URL in your browser (replace YOUR_CLIENT_ID with your actual client ID):

```
https://accounts.google.com/o/oauth2/v2/auth?
client_id=YOUR_CLIENT_ID&redirect_uri=http://dashboard-
server.local:5000/api/google/callback&response_type=code&scope=https://www.googleapis.com/auth/calend
```

5b. Sign in and approve the permissions

5c. You'll be redirected to a URL that looks like:

```
http://dashboard-server.local:5000/api/google/callback?code=4/0VERY_LONG_CODE_HERE
```

5d. Copy the code value (everything after code= and before any &)

5e. Exchange the code for tokens. Run this on your Pi (or any terminal):

```
curl -X POST https://oauth2.googleapis.com/token \
-H "Content-Type: application/x-www-form-urlencoded" \

-d "client_id=YOUR_CLIENT_ID" \

-d "client_secret=YOUR_CLIENT_SECRET" \

-d "code=THE_CODE_FROM_STEP_5D" \

-d "grant_type=authorization_code" \

-d "redirect_uri=http://dashboard-server.local:5000/api/google/callback"
```

5f. You'll get a JSON response like:

```
{
  "access_token": "ya29.a0...",

  "expires_in": 3599,

  "refresh_token": "1//0e...",

  "scope": "https://www.googleapis.com/auth/calendar.readonly",

  "token_type": "Bearer"
}
```

5g. Copy the `refresh_token` value — this is what you put in your `.env` file. This token lasts indefinitely (as long as you don't revoke access).

Step 6: Rewrite the Auth Code

Open `server/googleCalendar.ts` and replace the entire `getAccessToken()` function (approximately lines 1-30) with the direct OAuth pattern from the "Replit Connector Rewrites" section above. Use these env vars:

- `process.env.GOOGLE_CLIENT_ID`
- `process.env.GOOGLE_CLIENT_SECRET`
- `process.env.GOOGLE_REFRESH_TOKEN`
- Token URL: `https://oauth2.googleapis.com/token`

Environment Variables

```
GOOGLE_CLIENT_ID=123456789-xxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxx

GOOGLE_REFRESH_TOKEN=1//0exxxxxxxxxxx
```

If the Refresh Token Stops Working

Google refresh tokens can expire if:

- You revoke access at `myaccount.google.com > Security > Third-party apps`
- The app stays in "Testing" mode and the token is older than 7 days
- You change the client secret

Fix: Redo steps 5a through 5g to get a new refresh token.

To avoid the 7-day expiry: In the OAuth consent screen, click "Publish App" to move from Testing to Production. Google may ask you to verify the app, but for personal use you can often skip that by staying under 100 users.

Integration 3: Gmail

Difficulty: Medium-Hard

Requires code change: YES (Replit connector rewrite)

Setup

Use the **same Google Cloud project** as Google Calendar. You just need to add the Gmail scope.

Step 1: Enable the Gmail API

1. Go to **APIs & Services**, then **Library**
2. Search for "Gmail API"
3. Click it, then click **Enable**

Step 2: Add Gmail Scope

1. Go to **OAuth consent screen**
2. Click "Edit App"
3. Go to "Scopes" page
4. Add: `https://www.googleapis.com/auth/gmail.readonly`
5. If you also need to send emails via Gmail: add `https://www.googleapis.com/auth/gmail.send`
6. Save

Step 3: Get a New Refresh Token

Because you added a new scope, you need a new refresh token that includes both Calendar and Gmail:

```
https://accounts.google.com/o/oauth2/v2/auth?  
client_id=YOUR_CLIENT_ID&redirect_uri=http://dashboard-  
server.local:5000/api/google/callback&response_type=code&scope=https://www.googleapis.com/auth/calend
```

Then exchange the code for tokens using the same curl command as step 5e in the Calendar section.

Note: You can use the SAME client ID, secret, and refresh token for both Calendar and Gmail — they share the same Google project. Just use the same env vars in both files:

```
GOOGLE_CLIENT_ID=same_as_calendar
```

```
GOOGLE_CLIENT_SECRET=same_as_calendar
```

```
GOOGLE_REFRESH_TOKEN=new_token_with_all_scopes
```

Step 4: Rewrite the Auth Code

Open `server/gmail.ts` and replace the `getAccessToken()` function with the same direct OAuth pattern. Use the same env vars as Google Calendar.

Integration 4: Spotify

Difficulty: Medium

Requires code change: SMALL (just update the redirect URI logic)

Spotify already uses direct OAuth (not Replit connectors), so it mostly works. You just need to update how the redirect URI is determined.

Step 1: Create a Spotify App

1. Go to **developer.spotify.com/dashboard**
2. Click "Create App"
3. Fill in:
 - App name: "Dashboard"
 - App description: "Home dashboard music player"
 - Redirect URI: `http://dashboard-server.local:5000/api/spotify/callback`
(also add your Pi's IP version)
4. Check the "Web API" box
5. Click "Save"

Step 2: Get Client ID and Secret

1. In your app's dashboard, you'll see the **Client ID**
2. Click "Show client secret" to see the **Client Secret**
3. Copy both

Step 3: Small Code Change

In `server/spotify.ts`, find these lines (around line 33 and line 66):

```
const host = process.env.REPLIT_DOMAINS?.split(',')[0] || process.env.REPL_SLUG + '.' +  
process.env.REPL_OWNER + '.repl.co';  
  
const redirectUri = https://${host}/api/spotify/callback;
```

Replace with:

```
const host = process.env.DEPLOYED_APP_URL?.replace(/^https?:\/\//, '') || 'dashboard-  
server.local:5000';  
  
const redirectUri = http://${host}/api/spotify/callback;
```

Step 4: Authorize Spotify

1. Start the app on your Pi
2. Open `http://PI_IP:5000/api/spotify/login` in a browser
3. Sign in to Spotify and approve
4. You'll be redirected back — the app saves the tokens to `.spotify-token.json` automatically
5. That's it — Spotify is connected

Environment Variables

```
SPOTIFY_CLIENT_ID=your_spotify_client_id  
SPOTIFY_CLIENT_SECRET=your_spotify_client_secret
```

If Spotify Disconnects

The app auto-refreshes Spotify tokens using the saved refresh token in `.spotify-token.json`. If that file gets deleted or corrupted:

1. Delete `.spotify-token.json` from the app directory
2. Visit `http://PI_IP:5000/api/spotify/login` again
3. Re-authorize

Integration 5: Microsoft OneDrive

Difficulty: Hard

Requires code change: YES (Replit connector rewrite)

Step 1: Register an Azure App

1. Go to **portal.azure.com**
2. Search for "App registrations" in the top search bar
3. Click "New registration"
4. Fill in:
 - Name: "Dashboard"
 - Supported account types: "Accounts in any organizational directory and personal Microsoft accounts"
 - Redirect URI: Platform = "Web", URI = `http://dashboard-server.local:5000/api/microsoft/callback`
5. Click "Register"

Step 2: Get Client ID

After registering, you'll see the **Application (client) ID** on the overview page. Copy it.

Step 3: Create a Client Secret

1. Go to **Certificates & secrets** in the left menu
2. Click "New client secret"
3. Description: "Dashboard App"
4. Expiry: Choose "24 months" (you'll need to renew after that)
5. Click "Add"
6. **Copy the Value immediately** — you can't see it again after leaving this page

Step 4: Add API Permissions

1. Go to **API permissions** in the left menu
2. Click "Add a permission"
3. Choose "Microsoft Graph"
4. Choose "Delegated permissions"
5. Add these permissions:
 - `Files.ReadWrite` (OneDrive file access)
 - `Files.ReadWrite.All` (full OneDrive access)
 - `User.Read` (basic profile)
6. Click "Add permissions"
7. If you see "Grant admin consent", click it (if you're the admin)

Step 5: Get a Refresh Token

- 5a. **Open this URL** (replace YOUR_CLIENT_ID):

```
https://login.microsoftonline.com/common/oauth2/v2.0/authorize?
client_id=YOUR_CLIENT_ID&response_type=code&redirect_uri=http://dashboard-
server.local:5000/api/microsoft/callback&scope=Files.ReadWrite%20Files.ReadWrite.All%20User.Read%20of
```

5b. Sign in with your Microsoft account and approve

5c. Copy the `code` from the redirect URL

5d. Exchange for tokens:

```
curl -X POST https://login.microsoftonline.com/common/oauth2/v2.0/token \
  -H "Content-Type: application/x-www-form-urlencoded" \

  -d "client_id=YOUR_CLIENT_ID" \

  -d "client_secret=YOUR_CLIENT_SECRET" \

  -d "code=THE_CODE" \

  -d "grant_type=authorization_code" \

  -d "redirect_uri=http://dashboard-server.local:5000/api/microsoft/callback" \

  -d "scope=Files.ReadWrite%20Files.ReadWrite.All%20User.Read%20offline_access"
```

5e. Copy the `refresh_token` from the response

Step 6: Rewrite the Auth Code

Open `server/onedrive.ts` and replace the `getAccessToken()` function. For Microsoft, the token URL is different:

```
let cachedAccessToken: string | null = null;
let tokenExpiresAt: number = 0;

async function getAccessToken(): Promise<string> {

  if (cachedAccessToken && Date.now() < tokenExpiresAt - 5 * 60 * 1000) {

    return cachedAccessToken;

  }
```

```
const clientId = process.env.MICROSOFT_CLIENT_ID;

const clientSecret = process.env.MICROSOFT_CLIENT_SECRET;

const refreshToken = process.env.MICROSOFT_REFRESH_TOKEN;

if (!clientId || !clientSecret || !refreshToken) {

    throw new Error('Microsoft OAuth credentials not configured');

}

const response = await fetch('https://login.microsoftonline.com/common/oauth2/v2.0/token', {

    method: 'POST',

    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },

    body: new URLSearchParams({

        client_id: clientId,

        client_secret: clientSecret,

        refresh_token: refreshToken,

        grant_type: 'refresh_token',

        scope: 'Files.ReadWrite Files.ReadWrite.All User.Read offline_access',

    }),

});

if (!response.ok) {

    const error = await response.text();

    throw new Error( Failed to refresh Microsoft token: ${error} );
```

```

    }

    const data = await response.json();

    cachedAccessToken = data.access_token;

    tokenExpiresAt = Date.now() + (data.expires_in * 1000);

    // Microsoft sometimes returns a new refresh token – save it

    if (data.refresh_token && data.refresh_token !== refreshToken) {

        console.log('[OneDrive] Got new refresh token – update your .env MICROSOFT_REFRESH_TOKEN');

    }

    return cachedAccessToken!;

}

```

Environment Variables

```

MICROSOFT_CLIENT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx
MICROSOFT_CLIENT_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

MICROSOFT_REFRESH_TOKEN=0.AAAA...very_long_token

```

Important: Microsoft Refresh Token Rotation

Microsoft sometimes issues a NEW refresh token when you use the old one. If this happens, the old one stops working. The code above logs a warning when this happens. When you see that log message, update your .env file with the new token.

To handle this automatically, you could save the new token to a file:

```
if (data.refresh_token && data.refresh_token !== refreshToken) {  
  fs.writeFileSync('/opt/dashboard/.microsoft-refresh-token', data.refresh_token);  
  
}
```

Integration 6: Microsoft Outlook Calendar

Difficulty: Hard

Requires code change: YES (Replit connector rewrite)

Setup

Use the **same Azure app** as OneDrive. Just add Outlook permissions.

Step 1: Add Calendar Permissions

1. Go to your Azure app, then **API permissions**

2. Add these Delegated permissions:

- Calendars.Read
- Calendars.ReadWrite (if you need to create events)
- Mail.Read (if you also need email access)
- Mail.ReadWrite (if you need to move emails to folders)

3. Click "Grant admin consent" if available

Step 2: Get a New Refresh Token

Because you added new scopes, get a new token with ALL Microsoft scopes:

```
https://login.microsoftonline.com/common/oauth2/v2.0/authorize?  
client_id=YOUR_CLIENT_ID&response_type=code&redirect_uri=http://dashboard-  
server.local:5000/api/microsoft/callback&scope=Files.ReadWrite%20Files.ReadWrite.All%20Calendars.Read
```

Exchange the code for tokens using the same curl command as OneDrive step 5d, but with the expanded scope.

Step 3: Rewrite the Auth Code

Open `server/outlookCalendar.ts` and replace `getOutlookAccessToken()` with the same pattern as OneDrive. You can use the **same** env vars (`MICROSOFT_CLIENT_ID`, `MICROSOFT_CLIENT_SECRET`, `MICROSOFT_REFRESH_TOKEN`) since it's the same Azure app.

Important: One Token for Both

Since OneDrive and Outlook use the same Azure app, you only need ONE set of Microsoft credentials and ONE refresh token. Both files can use the same env vars. Just make sure the refresh token was obtained with ALL the scopes (Files + Calendar + Mail).

Integration 7: OpenAI (TTS)

Difficulty: Easy

Requires code change: YES (env var name change)

OpenAI generates the text-to-speech audio for the cat washroom reading system. On Replit, it uses the Replit AI Integrations system. On the Pi, you need your own API key.

Step 1: Get an API Key

1. Go to **platform.openai.com**
2. Click your profile icon, then "API keys" (or go to `platform.openai.com/api-keys`)
3. Click "Create new secret key"
4. Name it "Dashboard TTS"
5. Copy the key

Step 2: Add to .env

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Step 3: Code Change

Open `server/replit_integrations/audio/client.ts` and find:

```
apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
```

Change to:

```
apiKey: process.env.OPENAI_API_KEY,
```

Also check `server/routes.ts` for any references to `AI_INTEGRATIONS_OPENAI_API_KEY` and change them the same way.

Cost

OpenAI TTS costs about \$0.015 per 1,000 characters. A typical 10-page PDF is about 5,000 characters, so about \$0.075 (7.5 cents) per reading. For a full semester of ~50 readings, that's about \$3-4 total.

The app has fallback TTS engines (Edge TTS and `espeak-ng`) that are free, so if you run out of OpenAI credits, readings will still work — they'll just sound slightly different.

Integration 8: Resend (Email Sending)

Difficulty: Easy

Requires code change: No

Resend sends reminder emails (task due dates, daily digests) from `reminders@uni-cal.app`.

Step 1: Create a Resend Account

1. Go to **resend.com** and sign up
2. Free tier gives you 100 emails/day — more than enough

Step 2: Set Up a Domain (Optional but Recommended)

If you have your own domain:

1. In Resend dashboard, go to "Domains"
2. Add your domain and set up the DNS records they give you
3. This lets you send from a custom email address

If you don't have a domain, you can send from Resend's default: `onboarding@resend.dev` (limited to your verified email only).

Step 3: Get API Key

1. In Resend dashboard, go to "API Keys"
2. Click "Create API Key"
3. Name: "Dashboard"
4. Permission: "Full access"
5. Copy the key

Environment Variables

```
RESEND_API_KEY=re_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Integration 9: Second Google Account

Difficulty: Medium

Requires code change: SMALL

This is for your partner's work shift calendar. It already uses direct OAuth (not Replit connectors), so it mostly works on the Pi.

Setup

1. Use the same Google Cloud project as your main calendar
2. Make sure the redirect URI includes your Pi's address
3. The app has built-in OAuth flow at `/api/second-google/auth`

Code Change

In `server/secondGoogleAccount.ts`, find the redirect URI construction (around line 21):

```
const domain = process.env.REPLIT_DEV_DOMAIN || process.env.REPLIT_DOMAINS?.split(',')[0];
```

Replace with:

```
const domain = process.env.DEPLOYED_APP_URL?.replace(/^https?:\/\//, '') || 'dashboard-server.local:5000';
```

Environment Variables

```
GOOGLE_SECOND_ACCOUNT_CLIENT_ID=same_as_main_or_different
GOOGLE_SECOND_ACCOUNT_CLIENT_SECRET=same_as_main_or_different
```

Integration 10: Third Google Account

Difficulty: Medium

Requires code change: SMALL (same as second account)

Same setup as the Second Account. In `server/thirdGoogleAccount.ts`, make the same redirect URI fix.

Integration 11: Object Storage

Difficulty: Hard

Requires code change: YES (full replacement)

On Replit, file uploads (PDFs, TTS audio) are stored in Replit's Object Storage service. On a Pi, this doesn't exist.

Options (Pick One)

Option A: Local File System (Simplest)

Replace Object Storage calls with simple file system operations. Store files in `/opt/dashboard/uploads/`.

This is the easiest approach for self-hosting. You'd need to:

1. Create `/opt/dashboard/uploads/public/` and `/opt/dashboard/uploads/private/`
2. Rewrite the upload endpoints to use `fs.writeFileSync` instead of Object Storage
3. Serve the public files via Express static middleware

Option B: MinIO (S3-Compatible, More Robust)

MinIO is a free, self-hosted object storage that works like Amazon S3. Install it on the Pi:

```
wget https://dl.min.io/server/minio/release/linux-arm64/minio
chmod +x minio

sudo mv minio /usr/local/bin/
```

Start MinIO

```
mkdir -p /opt/minio-data

minio server /opt/minio-data --console-address ":9001"
```

Then update the object storage client code to use MinIO's S3-compatible API.

Option C: Keep Replit for Storage

If you keep running the app on Replit (even just for storage), you can keep the Object Storage as-is.

ChatGPT Prompt for Object Storage Rewrite

[Paste the intro statement]

I need to replace Replit Object Storage with local file system storage on my Raspberry Pi. The app currently uses object storage for:

1. Storing uploaded PDF files (via /api/course-week-upload endpoint)
2. Storing generated TTS audio files (MP3s, served via /api/tts-audio/ endpoint)

The current code uses these files for object storage operations:

- server/replit_integrations/object_storage/objectStorage.ts
- server/replit_integrations/object_storage/routes.ts

I want to replace all object storage calls with simple fs.readFileSync/fs.writeFileSync operations, storing files in /opt/dashboard/uploads/. I need the /api/tts-audio/ endpoint to keep working so the Nest speaker can stream audio files.

Please show me what to change.

App Security

SITE_PASSWORD

This password protects the dashboard from unauthorized access. Anyone who visits the URL gets a login prompt.

```
SITE_PASSWORD=your_chosen_password
```

If you leave this empty, the dashboard has no login protection (fine if it's only accessible on your local network).

SESSION_SECRET

This is a random string used to sign browser session cookies. Generate one with:

```
openssl rand -hex 32
```

```
SESSION_SECRET=a1b2c3d4e5f6...random_hex_string
```

Complete .env Template

Here is every environment variable your app uses. Copy this entire block into `/opt/dashboard/.env` and fill in the values:

===== DATABASE =====

`DATABASE_URL=postgresql://dashboard:YOUR_DB_PASSWORD@localhost:5432/dashboard_db`

===== APP =====

`DEPLOYED_APP_URL=http://dashboard-server.local:5000`

`PORT=5000`

`SITE_PASSWORD=your_dashboard_login_password`

`SESSION_SECRET=generate_with_openssl_rand_hex_32`

`NODE_ENV=production`

===== HOME ASSISTANT =====

`HOME_ASSISTANT_TOKEN=eyJ0eX...your_ha_long_lived_token`

`HOME_ASSISTANT_URL_OVERRIDE=https://your-ha-url.ui.nabu.casa`

===== GOOGLE (Calendar + Gmail) =====

`GOOGLE_CLIENT_ID=123456789-xxxx.apps.googleusercontent.com`

GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxx

GOOGLE_REFRESH_TOKEN=1//0xxxxxxxxxxxxx

===== GOOGLE (Second Account - Partner Shifts) =====

GOOGLE_SECOND_ACCOUNT_CLIENT_ID=same_or_different_client_id

GOOGLE_SECOND_ACCOUNT_CLIENT_SECRET=same_or_different_client_secret

(Refresh token stored in database after OAuth flow)

===== SPOTIFY =====

SPOTIFY_CLIENT_ID=your_spotify_client_id

SPOTIFY_CLIENT_SECRET=your_spotify_client_secret

(Refresh token stored in .spotify-token.json after OAuth flow)

===== MICROSOFT (OneDrive + Outlook)

=====

MICROSOFT_CLIENT_ID=xxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx

MICROSOFT_CLIENT_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

MICROSOFT_REFRESH_TOKEN=0.AAAA...very_long_token

===== OPENAI (TTS) =====

OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxx

===== RESEND (Email) =====

RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxxxxx

===== TIMEZONE =====

TZ=America/Toronto

Testing Each Integration

After setting everything up, test each integration one at a time:

Home Assistant


```
curl -s http://localhost:5000/api/version
```

Should return the app version

```
curl -s http://localhost:5000/api/weather
```

Should return weather data from HA

Google Calendar

```
curl -s http://localhost:5000/api/events
```

Should return calendar events (might need auth cookie – test from the dashboard)

Spotify

```
curl -s http://localhost:5000/api/spotify/status
```

Should show connected: true after authorizing

OneDrive

```
curl -s http://localhost:5000/api/onedrive/files
```

Should return file listing (might need auth cookie)

TTS (OpenAI)

- Turn on the cat lights — the app should generate TTS and you should hear the prompt

Resend

- Create a task with a reminder — you should get an email when it's due

ChatGPT Prompts for Integration Help

General OAuth Issues

[Paste the intro statement from the Troubleshooting Guide]

I'm setting up OAuth 2.0 for [Google/Microsoft/Spotify] on my self-hosted Pi. I've rewritten the `getAccessToken()` function to use direct refresh token flow instead of Replit connectors.

When I try to [describe action], I get this error: [paste error].

My current `getAccessToken()` code is:

[paste the function]

My environment variables (redacted):

- `CLIENT_ID`: [present/missing]

- `CLIENT_SECRET`: [present/missing]

- `REFRESH_TOKEN`: [present/missing]

The token URL I'm using is: [paste URL]

The scopes I requested were: [list scopes]

Token Exchange Issues

[Paste the intro statement]

I'm trying to exchange an OAuth authorization code for tokens. I have:

- Service: [Google/Microsoft/Spotify]
- Client ID: [present]
- Client Secret: [present]
- Authorization Code: [present, starts with...]
- Redirect URI: [paste exact URI]

When I run the curl command to exchange, I get: [paste the response]

Here is the exact curl command I ran (with secrets redacted):

[paste command]

Refresh Token Expired

[Paste the intro statement]

My [Google/Microsoft/Spotify] refresh token has stopped working. The error I get when trying to refresh is: [paste error].

This token was working [yesterday/last week/a month ago]. I [did/did not] revoke access, [did/did not] change the client secret, [did/did not] remove test users.

How do I get a new refresh token? Please give me the exact URL to open in my browser and the exact curl command to exchange the code.

Replit Connector Rewrite Help

[Paste the intro statement]

I need to rewrite the authentication code in [file name] to work without Replit connectors. Here is the current file:

[paste the first 60 lines]

Currently it uses Replit's connector API (REPLIT_CONNECTORS_HOSTNAME, REPL_IDENTITY) which doesn't exist on my self-hosted Pi. I need it to use direct OAuth 2.0 refresh token flow with environment variables (CLIENT_ID, CLIENT_SECRET, REFRESH_TOKEN).

Please rewrite only the authentication function. Keep everything else the same. The function should cache the access token and refresh it automatically before it expires.

Complete Troubleshooting Guide — Self-Hosting on Raspberry Pi

This guide covers every section of the Self-Hosting Guide and gives you:

1. What can go wrong in each step
 2. How to diagnose the problem
 3. How to fix it
 4. A ready-to-copy ChatGPT prompt if you need AI help
-

ChatGPT Introduction Statement (Copy This First, Every Time)

Whenever you need to ask ChatGPT for help with this project, **paste this block first** before describing your problem. It gives ChatGPT enough context to understand what you're working with:

> **Copy everything between the lines below:**

I am self-hosting a full-stack academic task management web application on a Raspberry Pi 5 (8GB RAM) running Raspberry Pi OS Lite (64-bit, no desktop environment).

The app is built with:

- Backend: Node.js 20 + Express.js + TypeScript

- Frontend: React + Vite + TypeScript + Tailwind CSS + shadcn/ui components
- Database: PostgreSQL (local, on the same Pi)
- ORM: Drizzle ORM (schema defined in shared/schema.ts, pushed with "npm run db:push")
- Build: The app is built with "npm run build" which compiles TypeScript and bundles the frontend. The production server runs with "node dist/index.js" on port 5000.
- Process Manager: systemd service called "dashboard" (runs as user "pi", working directory /opt/dashboard)

The app integrates with:

- Home Assistant (via REST API using a long-lived access token) for smart home automation, media player control, and sensor data
- Google Calendar API (OAuth 2.0) for calendar events
- Gmail API (OAuth 2.0) for email processing
- Spotify Web API (OAuth 2.0) for music playback control
- Microsoft Graph API (OAuth 2.0) for OneDrive file access and Outlook calendar
- OpenAI TTS API for text-to-speech audio generation (with Edge TTS and espeak-ng as fallbacks)

The app has a "cat washroom study reading system" that:

- Converts PDF files to text, chunks them, generates TTS audio
- Plays audio on a Google Nest speaker via Home Assistant's media_player service
- Syncs a follow-along text display on a Fire Tablet and Samsung TV via a Fire Stick
- Is triggered by Home Assistant webhooks (light switches, buttons, voice commands)
- Uses webhook endpoints like /api/webhook/cat-lights, /api/webhook/voice-command, etc.

The app serves both the API and the React frontend from the same Express server on port 5000. In production, Vite builds static files that Express serves.

The main server file is `server/routes.ts` (~17,000 lines). The database schema is in `shared/schema.ts`. The frontend entry point is `client/src/App.tsx`.

Environment variables are stored in `/opt/dashboard/.env` and loaded via `systemd`'s `EnvironmentFile` directive.

Here is my problem:

> Then describe your specific problem after "Here is my problem:"

Section-by-Section Troubleshooting

Section 1: Flashing the OS (Raspberry Pi Imager)

Problem: "The imager won't detect my microSD card"

Diagnosis:

On your computer (not the Pi), check if the card is detected

Windows: Open Disk Management (right-click Start then Disk Management)

Mac: Open Disk Utility

Linux: Run:

```
lsblk
```

Fixes:

- Try a different USB port
- Try a different card reader
- The card might be locked — check the tiny switch on the side of the SD adapter (slide it UP toward the label end)
- Try formatting the card first with SD Card Formatter from [sdcard.org/downloads/formatter/](https://www.sdcard.org/downloads/formatter/)

Problem: "Flash completed but verification failed"

Fixes:

- Re-download the Raspberry Pi Imager (your download might be corrupted)
- Try a different microSD card — verification failures often mean the card is defective
- Make sure you're not running out of space on the card

ChatGPT prompt addition:

I'm using Raspberry Pi Imager to flash Raspberry Pi OS Lite (64-bit) to a microSD card for a Raspberry Pi 5. The flash completes but verification fails. I've tried [describe what you tried]. Here is the error message: [paste error]

Section 2: First Boot and SSH

Problem: "ssh: connect to host dashboard-server.local port 22: Connection refused"

Diagnosis:

Try pinging the hostname

```
ping dashboard-server.local
```

If that doesn't work, find the Pi's IP from your router's admin page

(usually 192.168.1.1 or 192.168.0.1 in a browser)

Then try:

```
ssh pi@192.168.1.XXX
```

Fixes:

- Wait longer — first boot can take up to 5 minutes
- Make sure ethernet cable is plugged in before powering on
- Check that SSH was enabled in the Imager settings (if not, you need to re-flash)
- If using Wi-Fi only: the Pi might not have connected — re-flash with correct Wi-Fi credentials

Problem: "Permission denied (publickey)" or wrong password

Fixes:

- Make sure you're using the password you set in the Imager (not the default "raspberry")
- If you forgot the password, re-flash the card with new settings
- If you get publickey errors, force password auth:

```
ssh -o PreferredAuthentications=password pi@dashboard-server.local
```

ChatGPT prompt addition:

I flashed Raspberry Pi OS Lite (64-bit) to a Pi 5 using Raspberry Pi Imager with SSH enabled, username "pi", and a custom password. When I try to SSH in, I get this error: [paste error]. I am connected via [ethernet/wifi]. My router shows [does it show the Pi or not?].

Section 3: Installing System Dependencies

Problem: "E: Unable to locate package nodejs" or Node.js version is wrong

Diagnosis:

```
node --version
```

Should show v20.x.x

Fixes:

Remove any old Node.js

```
sudo apt remove -y nodejs
```

```
sudo rm -f /etc/apt/sources.list.d/nodesource.list
```

Re-add NodeSource repository

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

Verify

```
node --version
```

```
npm --version
```

Problem: "espeak-ng: command not found" after installing

Fix:

```
sudo apt install -y espeak-ng
```

If still not found:

```
which espeak-ng
```

Should show /usr/bin/espeak-ng

Problem: "pip3 install edge-tts" fails

Fixes:

Make sure pip is installed

```
sudo apt install -y python3-pip python3-venv
```

Try with --break-system-packages flag (required on newer Debian/Pi OS)

```
pip3 install edge-tts --break-system-packages
```

Verify

```
edge-tts --version
```

ChatGPT prompt addition:

I'm installing dependencies on a Raspberry Pi 5 running Raspberry Pi OS Lite (64-bit, Debian Bookworm based). I ran the following command: [paste command]. I got this error: [paste error]. My current versions are: Node [paste node --version], Python [paste python3 --version], pip [paste pip3 --version].

Section 4: Setting Up PostgreSQL

Problem: "psql: error: connection refused" or "role does not exist"

Diagnosis:

Check if PostgreSQL is running

```
sudo systemctl status postgresql
```

If it's not running:

```
sudo systemctl start postgresql
```

```
sudo systemctl enable postgresql
```

Fixes:

If the user already exists and you need to reset the password:

```
sudo -u postgres psql -c "ALTER USER dashboard WITH PASSWORD 'your_new_password';"
```

If the database already exists:

```
sudo -u postgres psql -c "DROP DATABASE dashboard_db;"
```

```
sudo -u postgres psql -c "CREATE DATABASE dashboard_db OWNER dashboard;"
```

Problem: "FATAL: Peer authentication failed for user 'dashboard'"

Fix: Edit the PostgreSQL auth config:

```
sudo nano /etc/postgresql/*/main/pg_hba.conf
```

Find the line that says:

```
local    all        all        peer
```

Change `peer` to `md5` :

```
local    all        all        md5
```

Then restart PostgreSQL:

```
sudo systemctl restart postgresql
```

ChatGPT prompt addition:

I'm setting up PostgreSQL on a Raspberry Pi 5 (Raspberry Pi OS Lite, Debian Bookworm). I created a user called "dashboard" and a database called "dashboard_db". When I try to connect, I get this error: [paste error]. Here is my pg_hba.conf: [paste relevant lines]. PostgreSQL version: [paste psql --version].

Section 5: Transferring the Code

Problem: "scp: command not found" (on Windows)

Fix: Use one of these instead:

- **PowerShell** (Windows 10+): scp should work natively in PowerShell
- **WinSCP**: Free graphical tool — download from winscp.net. Just drag and drop files.
- **FileZilla**: Another free option — set protocol to SFTP, port 22

Problem: "npm install" fails with errors

Diagnosis:

Check available memory

```
free -h
```

Check disk space

```
df -h
```

Fixes:

If out of memory (Pi 5 8GB should be fine, but just in case):

Create a swap file

```
sudo fallocate -l 2G /swapfile
```

```
sudo chmod 600 /swapfile
```

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

Then retry

```
npm install
```

If you get permission errors:

```
sudo chown -R pi:pi /opt/dashboard
```

```
npm install
```

Problem: "npm install" hangs or takes forever

Fixes:

- This is normal on a Pi for the first install — it can take 5-10 minutes
- If it's been more than 15 minutes, press Ctrl+C and try again

- Make sure you have a stable internet connection

ChatGPT prompt addition:

I transferred my Node.js project to a Raspberry Pi 5 at /opt/dashboard. When I run "npm install", I get this error: [paste error]. My Node version is [paste node --version]. My npm version is [paste npm --version]. Available memory: [paste output of "free -h"]. Disk space: [paste output of "df -h /opt/dashboard"].

Section 6: Environment Variables (.env file)

Problem: "The app starts but can't connect to the database"

Diagnosis:

Check your DATABASE_URL format

```
grep DATABASE_URL /opt/dashboard/.env
```

It should look exactly like this (with your actual password):

DATABASE_URL=postgresql://dashboard:YOUR_PASSWORD@:

Common mistakes:

- Password contains special characters that need escaping (wrap the whole URL in quotes, or URL-encode special chars like @ # % etc.)
- Typo in the database name (dashboard_db)
- PostgreSQL isn't running: `sudo systemctl status postgresql`

Problem: "Home Assistant webhooks aren't working"

Diagnosis:

Test the HA connection from the Pi

```
curl -s -H "Authorization: Bearer YOUR_HA_TOKEN" \  
  
https://YOUR_HA_URL/api/ | head -20
```

You should see: {"message": "API running."}

Fixes:

- Make sure your HA long-lived access token is correct (generate a new one from HA, go to Profile, then Long-Lived Access Tokens)
- If using Nabu Casa URL: make sure your HA subscription is active
- If using local URL: make sure the Pi can reach HA on your network

ChatGPT prompt addition:

```
I'm setting up environment variables for my self-hosted Node.js app on a Raspberry Pi 5. The .env  
file is at /opt/dashboard/.env. When the app starts, I see this error in the logs: [paste error].  
Here are the relevant environment variables (with secrets redacted):  
DATABASE_URL=postgresql://dashboard:*@localhost:5432/dashboard_db, HOME_ASSISTANT_TOKEN=  
[present/missing], DEPLOYED_APP_URL=[paste value].
```

Section 7: OAuth Setup (Google, Spotify, Microsoft)

This is the hardest part. Here's how to troubleshoot each one.

Google (Calendar + Gmail)

Problem: "Error 400: redirect_uri_mismatch"

Fix: **The redirect URI in your Google Cloud Console must EXACTLY match what the app sends. Go to:**

- 1. console.cloud.google.com, then your project, then Credentials, then your OAuth client**
- 2. Under "Authorized redirect URIs", add:**

```
http://dashboard-server.local:5000/api/google/callback
```

(or use the Pi's IP address: http://192.168.1.XXX:5000/api/google/callback)

Problem: "Access blocked: This app's request is invalid" or "Error 403"

Fix:

1. Make sure the Calendar API and Gmail API are enabled:

- Go to APIs and Services, then Library
- Search "Google Calendar API", then click Enable
- Search "Gmail API", then click Enable

2. If the app is in "Testing" mode, add your email as a test user:

- Go to OAuth consent screen, then Test users, then Add your Gmail address

Problem: "invalid_grant" error when using the refresh token

Fix: **The refresh token has expired or been revoked. You need to redo the OAuth flow:**

- 1. Open the authorization URL in a browser**
- 2. Sign in and approve**
- 3. Copy the new authorization code**
- 4. Exchange it for new tokens**
- 5. Update your .env with the new refresh token**

ChatGPT prompt addition:

```
I'm setting up Google OAuth 2.0 for my self-hosted Node.js app on a Raspberry Pi. I need Calendar API (read) and Gmail API (read) access. I created an OAuth 2.0 client in Google Cloud Console with redirect URI http://[my-pi-address]:5000/api/google/callback. When I try to authorize, I get this error: [paste error]. My OAuth consent screen is set to [Internal/External] and is in [Testing/Production] mode.
```

Spotify

Problem: "INVALID_CLIENT" error

Fix: **Double-check your Client ID and Client Secret at developer.spotify.com/dashboard. Make sure:**

- **The redirect URI is added: <http://dashboard-server.local:5000/api/spotify/callback>**
- **You're using the correct app's credentials (not a different app)**

Problem: "Insufficient client scope"

Fix: **When doing the OAuth flow, make sure you request all needed scopes. The app needs:**

```
user-read-playback-state user-modify-playback-state user-read-currently-playing playlist-read-private user-library-read user-read-recently-played streaming
```

ChatGPT prompt addition:

I'm setting up Spotify OAuth for my self-hosted Node.js app. I created a Spotify app at `developer.spotify.com` with redirect URI `http://[my-pi-address]:5000/api/spotify/callback`. When I try to `[authorize/play music/get playlists]`, I get this error: `[paste error]`. I have these scopes configured: `[list scopes]`.

Microsoft (OneDrive + Outlook)

Problem: "AADSTS50011: The reply URL specified in the request does not match"

Fix: In Azure Portal, go to App registrations, then your app, then Authentication:

- Add the redirect URI: `http://dashboard-server.local:5000/api/microsoft/callback`
- Make sure it's under "Web" platform (not SPA or Mobile)

Problem: "Insufficient privileges to complete the operation"

Fix: In Azure Portal, go to App registrations, then your app, then API permissions:

- Add these permissions (Delegated, not Application):
- Files.ReadWrite (OneDrive)
- Mail.Read (Outlook mail)
- Calendars.Read (Outlook calendar)
- Click "Grant admin consent" if you're the admin
- If you're not the admin, you need to ask your admin to approve

ChatGPT prompt addition:

I'm setting up Microsoft OAuth (Azure AD) for my self-hosted Node.js app. I registered an app in Azure Portal with redirect URI `http://[my-pi-address]:5000/api/microsoft/callback`. I need OneDrive (Files.ReadWrite), Outlook mail (Mail.Read), and Outlook calendar (Calendars.Read) access. When I try to `[authorize/access files/read calendar]`, I get this error: `[paste error]`. My app registration is set to `[single tenant/multi-tenant]`. The API permissions I've configured are: `[list them]`.

Section 8: Database Initialization (db:push)

Problem: "npm run db:push" fails with connection error

Diagnosis:

Make sure PostgreSQL is running

```
sudo systemctl status postgresql
```

Test the connection manually

```
psql -U dashboard -d dashboard_db -h localhost
```

Enter your password when prompted

If you get in, type \q to exit

Fixes:**

- Check DATABASE_URL in .env matches exactly
- Make sure PostgreSQL is listening on localhost:

```
sudo grep "listen_addresses" /etc/postgresql/*/main/postgresql.conf
```

Should show: listen_addresses = 'localhost'

Problem: "relation already exists" or migration conflicts

Fix:

Force push (drops and recreates tables – WARNING: destroys existing data)

```
npm run db:push --force
```

Or drop the database and start fresh:

```
sudo -u postgres psql -c "DROP DATABASE dashboard_db;"
```

```
sudo -u postgres psql -c "CREATE DATABASE dashboard_db OWNER dashboard;"
```

```
npm run db:push
```

ChatGPT prompt addition:

```
I'm using Drizzle ORM with PostgreSQL on a Raspberry Pi. My schema is defined in shared/schema.ts and I'm running "npm run db:push" to create the tables. I get this error: [paste error]. My DATABASE_URL is postgresql://dashboard:*@localhost:5432/dashboard_db. PostgreSQL is [running/not running]. Drizzle config is in drizzle.config.ts.
```

Section 9: Build and Test

Problem: "npm run build" fails with TypeScript errors

Diagnosis:

Run the build and save the full error output

```
npm run build 2>&1 | tee /tmp/build-errors.txt
```

Look at the first error (fix errors from top to bottom)

```
head -50 /tmp/build-errors.txt
```

Fixes:

- Most build errors are TypeScript type issues — they don't affect whether the app runs, but they block the build
- If you get "out of memory" during build:

Increase Node.js memory limit

```
export NODE_OPTIONS="--max-old-space-size=4096"
```

```
npm run build
```

Problem: "node dist/index.js" crashes immediately

Diagnosis:

Run it and see the error

```
node dist/index.js 2>&1 | head -50
```

Common causes:

- **Missing .env file or environment variables**
- **PostgreSQL not running**
- **Port 5000 already in use: run `sudo fuser -k 5000/tcp` then try again**
- **Missing npm packages: run `npm install` again**

Problem: "App starts but the page is blank in the browser"

Fixes:

- **Make sure you ran `npm run build` (not just dev mode — dev mode won't work in production)**
- **Check that dist/public exists and contains files: `ls dist/public/`**
- **Try accessing `http://PI_IP_ADDRESS:5000` (not localhost — you're not on the Pi)**

ChatGPT prompt addition:

```
I built my Node.js + React + Vite app on a Raspberry Pi 5 using "npm run build". The build [succeeded/failed with these errors: paste errors]. When I run "node dist/index.js", [it crashes with: paste error / it starts but: describe what happens]. I'm trying to access it from another device at http://[Pi address]:5000.
```

Section 10: systemd Service

Problem: "Service dashboard failed to start"

Diagnosis:

Check the status

```
sudo systemctl status dashboard
```

Check the logs

```
sudo journalctl -u dashboard -n 50 --no-pager
```

Check if the service file is correct

```
cat /etc/systemd/system/dashboard.service
```

Fixes:

- Make sure the paths are correct in the service file:
- `WorkingDirectory=/opt/dashboard` — this directory must exist
- `ExecStart=/usr/bin/node dist/index.js` — verify the node path with `which node`
- `EnvironmentFile=/opt/dashboard/.env` — this file must exist
- After editing the service file:

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart dashboard
```

Problem: "Service starts but keeps restarting"

Diagnosis:

Watch the logs live

```
sudo journalctl -u dashboard -f
```

Common causes:

- **The app crashes on startup (database connection, missing env vars)**
- **Port 5000 is already in use by another process**
- **File permissions:** run `sudo chown -R pi:pi /opt/dashboard`

ChatGPT prompt addition:

```
I set up a systemd service on a Raspberry Pi 5 to run my Node.js app. The service file is at /etc/systemd/system/dashboard.service. When I run "sudo systemctl start dashboard", [describe what happens]. Here is the output of "sudo systemctl status dashboard": [paste output]. Here are the last 50 lines of the journal: [paste output of "sudo journalctl -u dashboard -n 50 --no-pager"].
```

Section 11: Home Assistant Webhook Updates

Problem: "Webhooks worked on Replit but not from the Pi"

Diagnosis:

Test if HA can reach the Pi

From any computer on your network:

```
curl http://dashboard-server.local:5000/api/version
```

You should see a JSON response with the version

Fixes:

- In HA's configuration.yaml, update ALL rest_command URLs from:

```
https://home-view--bkh416.replit.app/api/webhook/...
```

to:

```
http://RASPBERRY_PI_IP:5000/api/webhook/...
```

- Use the Pi's static IP address (not hostname) for reliability — hostnames can be flaky on some networks

- Set a static IP on the Pi:

```
sudo nmcli con mod "Wired connection 1" ipv4.addresses 192.168.1.100/24
```

```
sudo nmcli con mod "Wired connection 1" ipv4.method manual
```

```
sudo nmcli con mod "Wired connection 1" ipv4.gateway 192.168.1.1
```

```
sudo nmcli con mod "Wired connection 1" ipv4.dns "8.8.8.8 8.8.4.4"
```

```
sudo nmcli con up "Wired connection 1"
```

(Replace the IP addresses with ones appropriate for your network.)

Problem: "HA automation fires but the app doesn't respond"

Diagnosis:

Check if the app is running

```
sudo systemctl status dashboard
```

Check the app logs for incoming webhook requests

```
sudo journalctl -u dashboard -f
```

Then trigger the automation in HA – you should see log lines starting with [Cat Lights] or [Shower Button] etc.

Fixes:

- If no log lines appear: HA isn't reaching the app. Check the URL in your rest_command.
- If log lines appear but show errors: the error message will tell you what's wrong

- Make sure there's no firewall blocking port 5000:

```
sudo ufw status
```

If active and port 5000 is not allowed:

```
sudo ufw allow 5000
```

ChatGPT prompt addition:

```
I'm connecting Home Assistant to my self-hosted Node.js app on a Raspberry Pi 5. HA is running on a separate device at [HA URL]. My app is at http://[Pi IP]:5000. I updated the rest_commands in configuration.yaml to point to the Pi. When I trigger the automation for [cat lights/shower button/voice command/etc.], [describe what happens – nothing, error, partial response]. Here are the HA rest_commands I configured: [paste relevant yaml]. Here are the app logs when I trigger it: [paste logs].
```

Section 12: Google Apps Script (Gmail Webhooks)

Problem: "Google Apps Script can't reach the Pi"

Explanation: **Google Apps Script runs in Google's cloud — it can't access devices on your home network directly. The Pi is behind your router's firewall.**

Solutions (pick one):

1. Cloudflare Tunnel (recommended, free):

Install cloudflared on the Pi

```
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm64  
-o /usr/local/bin/cloudflared
```

```
chmod +x /usr/local/bin/cloudflared
```

Authenticate (follow the URL it gives you)

```
cloudflared tunnel login
```

Create a tunnel

```
cloudflared tunnel create dashboard
```

Configure it

```
mkdir -p ~/.cloudflared
```

Then create `~/.cloudflared/config.yml` with:

```
tunnel: YOUR_TUNNEL_ID

credentials-file: /home/pi/.cloudflared/YOUR_TUNNEL_ID.json

ingress:

  - hostname: dashboard.yourdomain.com

    service: http://localhost:5000

  - service: http_status:404
```

Then run:

```
cloudflared tunnel run dashboard
```

2. Keep using Replit just for Gmail webhooks — **easiest option, no network changes needed**
3. Port forwarding — **forward port 5000 on your router to the Pi's IP (less secure, not recommended)**

ChatGPT prompt addition:

```
I have a Google Apps Script that sends webhook requests to my web app. The app was previously hosted on Replit (public URL) but I've moved it to a Raspberry Pi on my home network at http://192.168.x.x:5000. The Apps Script can no longer reach the app because it's behind my router. I want to set up [Cloudflare Tunnel / port forwarding / another solution] so Google's servers can reach my Pi. I [do/don't] have a domain name. My router is [brand/model if known].
```

Section 13: Touchscreen Setup

Problem: "The dashboard looks wrong on my 1920x720 screen"

Fixes:

- **Make sure the browser is in fullscreen mode (F11)**
- **Check the zoom level is 100% (Ctrl+0 to reset)**
- **The app is designed for exactly 1920x720 — other resolutions will look different**

Problem: "Touch events aren't working"

Fixes:

- **Most USB touchscreens work out of the box with Pi OS**
- **If touch is offset or inverted:**

List input devices

```
xinput list
```

Calibrate

```
sudo apt install -y xinput-calibrator
```

```
xinput_calibrator
```

General: App Crashes / Won't Start

Quick diagnostic checklist:

1. Is PostgreSQL running?

```
sudo systemctl status postgresql
```

2. Is the app service running?

```
sudo systemctl status dashboard
```

3. What do the logs say?

```
sudo journalctl -u dashboard -n 100 --no-pager
```

4. Can you start it manually?

```
cd /opt/dashboard
```

```
node dist/index.js
```

5. Is port 5000 free?

```
sudo fuser 5000/tcp
```

If something is using it:

```
sudo fuser -k 5000/tcp
```

6. Is the .env file readable?

```
cat /opt/dashboard/.env | head -5
```

7. Are node_modules installed?

```
ls /opt/dashboard/node_modules | head -5
```

8. Is the build up to date?

```
ls -la /opt/dashboard/dist/index.js
```

How to share logs with ChatGPT:

Save the last 200 lines of logs to a file

```
sudo journalctl -u dashboard -n 200 --no-pager > /tmp/dashboard-logs.txt
```

Then copy the contents and paste into ChatGPT after the intro statement

```
cat /tmp/dashboard-logs.txt
```

General: Database Issues

Problem: "relation 'tasks' does not exist" (or any table)

Tables weren't created. Run:

```
cd /opt/dashboard
```

```
npm run db:push
```

Problem: "column 'xyz' does not exist" (schema out of sync)

Schema changed but database wasn't updated. Run:

```
cd /opt/dashboard
```

```
npm run db:push
```

If that fails:

```
npm run db:push --force
```

WARNING: --force may drop and recreate tables, losing data. Back up first:

```
pg_dump -U dashboard dashboard_db > backup_before_push.sql
```

Problem: "ECONNREFUSED 127.0.0.1:5432" (can't connect to database)

PostgreSQL isn't running

```
sudo systemctl start postgresql
```

```
sudo systemctl enable postgresql
```

ChatGPT prompt addition:

I'm running PostgreSQL on a Raspberry Pi 5 for my Node.js app. The database is called "dashboard_db" and the user is "dashboard". I'm using Drizzle ORM with the schema in shared/schema.ts. When the app tries to [query/insert/connect], I get this error: [paste error]. PostgreSQL status: [paste output of "sudo systemctl status postgresql"]. I can/cannot connect manually with: `psql -U dashboard -d dashboard_db -h localhost`.

General: App Works But Something Specific Is Broken

If the app loads but a specific feature doesn't work, here's how to give ChatGPT the right context:

For calendar issues:

[Paste the intro statement from the top of this guide]

The calendar integration with [Google/Outlook/both] is not working. When I [open the dashboard/click on a date/try to create an event], I see [describe what happens]. The browser console shows: [open browser dev tools with F12, go to Console tab, paste any red errors]. The server logs show: [paste relevant lines from "sudo journalctl -u dashboard -n 50 --no-pager"].

For Spotify issues:

[Paste the intro statement from the top of this guide]

The Spotify integration is not working. When I [try to play/pause/see playlists], I see [describe what happens]. My Spotify OAuth tokens are [present in .env / missing]. The server logs show: [paste relevant lines].

For cat washroom / TTS issues:

[Paste the intro statement from the top of this guide]

The cat washroom study reading system is not working correctly. When I [turn on the cat lights / press the shower button / say a voice command], [describe what happens vs what should happen]. The server logs show: [paste the lines starting with "[Cat Lights]" or "[Shower Button]" or "[Voice Command]" from "sudo journalctl -u dashboard -f"].

The system uses these HA entities:

- Nest speaker: media_player.bathroom_speaker
- HA Voice: media_player.home_assistant_voice_097c38_media_player
- Fire Stick: media_player.fire_tv_172_24_0_88
- Samsung TV: media_player.tv_cat_wr
- Fire Tablet: media_player.tablet_cat (uses tablet-nav polling, NOT ADB)
- Echo speakers: media_player.echo_cat_left_am, echo_cat_right_am, echo_cat_washroom_middle
- Echo group: media_player.cat_washroom_media_group

For weather/ticker issues:

[Paste the intro statement from the top of this guide]

The weather/news ticker at the bottom of the dashboard is [not showing / showing wrong data / showing errors]. The ticker gets data from Home Assistant sensors (sensor.dashboard_ticker, sensor.dashboard_weather, sensor.dashboard_news). The server logs show: [paste relevant lines]. The HA sensors show: [check in HA Developer Tools then States].

Quick Reference: Useful Commands

| What You Want To Do | Command |

|-----|-----|

| Check if app is running | sudo systemctl status dashboard |

| View live logs | sudo journalctl -u dashboard -f |

| View last 100 log lines | sudo journalctl -u dashboard -n 100 --no-pager |

| Restart the app | `sudo systemctl restart dashboard` |

| Stop the app | `sudo systemctl stop dashboard` |

| Start the app | `sudo systemctl start dashboard` |

| Check PostgreSQL | `sudo systemctl status postgresql` |

| Check disk space | `df -h` |

| Check memory | `free -h` |

| Check what's using port 5000 | `sudo fuser 5000/tcp` |

| Kill whatever's on port 5000 | `sudo fuser -k 5000/tcp` |

| Back up the database | `pg_dump -U dashboard dashboard_db > backup.sql` |

| Rebuild the app | `npm run build` |

| Update database schema | `npm run db:push` |

| Check Pi temperature | `vcgencmd measure_temp` |

| Check Pi CPU usage | `top -bn1` (then press q to exit) |

How to Split routes.ts Into Two Files (Step-by-Step for Beginners)

Your `server/routes.ts` file is currently 17,239 lines long. That's huge — it slows down your code editor, makes it harder to find things, and uses more memory than it needs to. We're going to split it into two files:

1. `server/routes.ts` — The main app (tasks, calendar, settings, weather, ticker, files, Spotify, etc.) — roughly lines 1–7,430
2. `server/catWashRoutes.ts` — Everything related to the cat washroom study reading system, media playback, webhooks, and HA automations — roughly lines 7,430–17,239

After the split, the app will work exactly the same — nothing changes from the user's perspective. It's just organized better.

What You Need

- A code editor (VS Code, Notepad++, or even Notepad)
 - The `server/routes.ts` file open
 - About 30–45 minutes
-

Step 1: Find the Split Point

Open `server/routes.ts` and scroll to line 7,433** (or search for `SERVER_START_TIME`). You'll see something like this:

```
const SERVER_START_TIME = Date.now();

const SERVER_STARTUP_COOLDOWN_MS = 60 * 1000;


// ===== HA Connectivity Health Monitor =====
```

Everything from this line downward is the cat washroom / media / webhook code. That's what we're moving to the new file.

Step 2: Create the New File

Create a new file called `server/catWashRoutes.ts` . This file will contain all the cat washroom code.

Start the file with this header:

```
import type { Express } from "express";

import fs from "fs";


import path from "path";


import { storage } from "../storage";


import { db } from "../db";


import { sql, eq } from "drizzle-orm";


import { getWeekNumber, type FileRecord, appState, announcements } from "@shared/schema";


import { z } from "zod";


import { objectStorageClient } from "../replit_integrations/object_storage/objectStorage";


import { textToSpeech } from "../replit_integrations/audio/client";


import { sendEchoVoiceAnnouncement } from "../email";


import { listOneDriveItems, getOneDriveFile, getOneDriveItemByPath } from "../onedrive";


import * as spotifyApi from "../spotify";


import { torontoDate, torontoNow } from "../timezone";
```

(Don't worry if some of these imports aren't needed — TypeScript will tell you which ones to remove later. It's better to have too many than too few.)

Step 3: Copy the Shared Constants

Some constants are used by BOTH files (like the HA entity names). You need to either:

Option A (Easiest): Copy the constants block into BOTH files. Yes, it's duplicated, but it works and is simple.

Option B (Cleaner): Create a third file called `server/constants.ts`, put the constants there, and import from both files.

Option A — Just copy this block into the TOP of `catWashRoutes.ts` (after the imports):


```
const DEPLOYED_APP_URL = process.env.DEPLOYED_APP_URL || "https://home-view--bkh416.replit.app";
const HOME_ASSISTANT_URL = process.env.HOME_ASSISTANT_URL_OVERRIDE ||
"https://ec8ebfanqrqlsnmnggrdl4yzq2i8koah.ui.nabu.casa";

const tokenFromEnv = process.env.HOME_ASSISTANT_TOKEN || "";

const urlFromEnv = process.env.HOME_ASSISTANT_URL || "";

const HOME_ASSISTANT_TOKEN = tokenFromEnv.startsWith("eyJ") ? tokenFromEnv :
(urlFromEnv.startsWith("eyJ") ? urlFromEnv : tokenFromEnv);

const BATHROOM_ECHO_ENTITY = "media_player.bathroom_speaker";

const KITCHEN_ECHO_ENTITY = "media_player.echo_kitchen_studio_black_am";

const NEST_SPEAKER_ENTITY = "media_player.bathroom_speaker";

const CAT_WR_HA_VOICE_ENTITY = "media_player.home_assistant_voice_097c38_media_player";

const NON_ALEXA_ENTITIES = [NEST_SPEAKER_ENTITY, CAT_WR_HA_VOICE_ENTITY];

const MODULE_READING_PENDING = "input_boolean.module_reading_pending";

const MODULE_READING_CONFIRMED = "input_boolean.module_reading_confirmed";

const PARTNER_PHONE_ENTITY = "device_tracker.y_phone_app";

const HA_CLOUD_TTS_ENTITY = "tts.home_assistant_cloud";

const CAT_LIGHTS_ENTITY = "light.cat_lights";

const CAT_TV_ENTITY = "media_player.tv_cat_wr";

const FIRE_STICK_ADB_ENTITY = "media_player.fire_tv_172_24_0_88";

const CAT_WR_MEDIA_GROUP = "media_player.cat_washroom_media_group";

const CAT_ECHO_ENTITIES = [

  "media_player.echo_cat_left_am",
```

```

    "media_player.echo_cat_right_am",

    "media_player.echo_cat_washroom_middle",

];

const SPOTIFYPLUS_ENTITY = "media_player.spotifyplus_byhomeyyz";

const EVERYWHERE_GROUP_ENTITY = "media_player.byhome";

```

Option B — Create `server/constants.ts` :

```

// server/constants.ts — Shared constants used by routes.ts and catWashRoutes.ts

export const DEPLOYED_APP_URL = process.env.DEPLOYED_APP_URL || "https://home-view--bkh416.replit.app";

export const HOME_ASSISTANT_URL = process.env.HOME_ASSISTANT_URL_OVERRIDE ||
"https://ec8ebfanqrqlsnmnggrdl4yzq2i8koah.ui.nabu.casa";

const tokenFromEnv = process.env.HOME_ASSISTANT_TOKEN || "";

const urlFromEnv = process.env.HOME_ASSISTANT_URL || "";

export const HOME_ASSISTANT_TOKEN = tokenFromEnv.startsWith("eyJ") ? tokenFromEnv :
(urlFromEnv.startsWith("eyJ") ? urlFromEnv : tokenFromEnv);

export const BATHROOM_ECHO_ENTITY = "media_player.bathroom_speaker";

export const KITCHEN_ECHO_ENTITY = "media_player.echo_kitchen_studio_black_am";

export const NEST_SPEAKER_ENTITY = "media_player.bathroom_speaker";

export const CAT_WR_HA_VOICE_ENTITY = "media_player.home_assistant_voice_097c38_media_player";

export const NON_ALEXA_ENTITIES = [NEST_SPEAKER_ENTITY, CAT_WR_HA_VOICE_ENTITY];

export const MODULE_READING_PENDING = "input_boolean.module_reading_pending";

```

```

export const MODULE_READING_CONFIRMED = "input_boolean.module_reading_confirmed";

export const PARTNER_PHONE_ENTITY = "device_tracker.y_phone_app";

export const HA_CLOUD_TTS_ENTITY = "tts.home_assistant_cloud";

export const CAT_LIGHTS_ENTITY = "light.cat_lights";

export const CAT_TV_ENTITY = "media_player.tv_cat_wr";

export const FIRE_STICK_ADB_ENTITY = "media_player.fire_tv_172_24_0_88";

export const CAT_WR_MEDIA_GROUP = "media_player.cat_washroom_media_group";

export const CAT_ECHO_ENTITIES = [

    "media_player.echo_cat_left_am",

    "media_player.echo_cat_right_am",

    "media_player.echo_cat_washroom_middle",

];

export const SPOTIFYPLUS_ENTITY = "media_player.spotifyplus_byhomeyyz";

export const EVERYWHERE_GROUP_ENTITY = "media_player.byhome";

```

Then in BOTH `routes.ts` and `catWashRoutes.ts` , replace the constants with:

```

import { DEPLOYED_APP_URL, HOME_ASSISTANT_URL, HOME_ASSISTANT_TOKEN, BATHROOM_ECHO_ENTITY,
KITCHEN_ECHO_ENTITY, NEST_SPEAKER_ENTITY, CAT_WR_HA_VOICE_ENTITY, NON_ALEXA_ENTITIES,
MODULE_READING_PENDING, MODULE_READING_CONFIRMED, PARTNER_PHONE_ENTITY, HA_CLOUD_TTS_ENTITY,
CAT_LIGHTS_ENTITY, CAT_TV_ENTITY, FIRE_STICK_ADB_ENTITY, CAT_WR_MEDIA_GROUP, CAT_ECHO_ENTITIES,
SPOTIFYPLUS_ENTITY, EVERYWHERE_GROUP_ENTITY } from "../constants";

```

Step 4: Copy the Shared Helper Functions

These functions are used by the cat washroom code but are defined at the top of `routes.ts` . You need to copy them

into `catWashRoutes.ts` as well (or put them in a shared helpers file):

1. `haFetch` (around line 116) — Makes HTTP calls to Home Assistant with retries
2. `haServiceCall` (around line 145) — Calls HA services
3. `haServiceCallSafe` (around line 197) — Same but fire-and-forget (won't crash if it fails)
4. `processHACommandQueue` (around line 166) — Queues HA commands
5. `generateAndSaveTTSAudio` (around line 364) — Generates TTS audio files
6. `cleanTextForTTS` (around line 429) — Cleans text for speech
7. `getChunkWithSentenceBoundary` (around line 622) — Splits text at sentence boundaries
8. `stopTTSSession` (around line 639) — Stops a TTS session
9. `sendNextChunk` (around line 651) — Sends the next TTS chunk
10. `scheduleNextChunk` (around line 801) — Schedules the next chunk with timing
11. `parsePublicObjectPath` (around line 421) — Parses object storage paths
12. `formatLocalDate` (around line 217) — Formats dates for Toronto timezone

Copy all of these functions into `catWashRoutes.ts` after the constants.

Tip: Search for each function name in the file. If you see it's ONLY used in the cat washroom section (below line 7,433), you can REMOVE it from `routes.ts` after copying. If it's used in BOTH halves, leave it in both files.

Step 5: Move the Route Handlers

Now for the main event. In `catWashRoutes.ts`, create the function that will hold all the cat washroom routes:

```
export function registerCatWashRoutes(app: Express) {  
  // PASTE EVERYTHING FROM LINE 7,433 TO THE END OF THE registerRoutes FUNCTION HERE  
  
  // (That's roughly lines 7,433 through ~17,200)  
  
}
```

What to move (these are the sections):

| Section | What It Contains |

|-----|-----|

| HA Connectivity Health Monitor | Health checks for HA connection |

| Cat Washroom Webhooks | `/api/webhook/cat-lights`, `cat-shower-button`, `cat-lights-confirm`, `cat-wash-stop`, `cat-volume`, `cat-knob-press`, `voice-command` |

| Kitchen Volume Webhook | `/api/webhook/kitchen-volume` |

| Play Urgent PDF Webhook | `/api/webhook/play-urgent-pdf` |

| Shower/Media Routes | `/api/shower/*`, `/api/media/*`, `/api/kitchen/*` |

| TTS Routes | `/api/tts/*` |

| File Upload/Sync | `/api/course-week-upload`, `/api/files/prepare-audio`, `/api/shower/sync-onedrive` |

| Echo TTS | `/api/echo/tts` |

| Partner Status | `/api/partner-status`, `/api/ha/service` |

| Tablet Command Polling | `/api/tablet-command/*` |

| Playback Session | All the playback state management |

How to do the cut-and-paste:

1. In `routes.ts`, go to **line 7,433** (the `const SERVER_START_TIME = Date.now();` line)
 2. Select from there all the way down to the **last closing brace** of the `registerRoutes` function (but NOT the closing brace itself — that stays in `routes.ts`)
 3. **Cut** (Ctrl+X) all that selected text
 4. **Paste** (Ctrl+V) it inside the `registerCatWashRoutes` function in `catWashRoutes.ts`
 5. In `routes.ts`, the `registerRoutes` function should now end cleanly with just a `}` after the remaining routes
-

Step 6: Update `routes.ts` to Call the New File

At the **top** of `routes.ts`, add this import:

```
import { registerCatWashRoutes } from "../catWashRoutes";
```

Then at the **bottom** of the `registerRoutes` function (just before the final `}`), add:

```
// Register cat washroom & media routes from separate file
registerCatWashRoutes(app);
```

So the end of `routes.ts` should look like:

```
// ... last route in the main file ...

// Register cat washroom & media routes from separate file

registerCatWashRoutes(app);

}
```

Step 7: Test It

1. Save both files
2. Restart the app
3. If you get errors, they'll tell you exactly what's missing — usually a function or variable that one file needs but is defined in the other
4. Fix each error by either:
 - Copying the missing function/variable to the new file, OR
 - Moving it to `server/constants.ts` and importing from both

Common errors you might see:

| Error Message | What It Means | Fix |

|-----|-----|-----|

| Cannot find name 'haFetch' | The `haFetch` function is used in `catWashRoutes.ts` but wasn't copied over |
Copy the `haFetch` function into `catWashRoutes.ts` |

| Cannot find name 'HOME_ASSISTANT_URL' | A constant is missing | Copy the constant or import from
`constants.ts` |

| Cannot find name 'catWashPlaybackActive' | A state variable is missing | Copy the `let`
`catWashPlaybackActive = ...` line over |

| Module '"./catWashRoutes"' has no exported member... | You forgot to add `export` to the function |
Add `export` before function `registerCatWashRoutes` |

Step 8: Verify Everything Works

Test these things to make sure nothing broke:

1. **Dashboard loads** — Open the app in your browser
2. **Tasks work** — Create, edit, complete a task
3. **Calendar shows events** — Check Outlook and Google calendars

4. **Cat washroom webhook works** — Turn the cat lights on/off and check the logs
5. **Spotify player works** — Play/pause music
6. **PDF reader works** — Open a reading file
7. **Weather/ticker loads** — Check the bottom ticker bar

If all of those work, you're done!

Quick Summary

| Before | After |

|-----|-----|

| 1 file, 17,239 lines | 2 files, ~9,800 + ~7,400 lines each |

| `server/routes.ts` (everything) | `server/routes.ts` (main app) + `server/catWashRoutes.ts` (cat washroom + media) |

What goes where:

| File | Contains |

|-----|-----|

| `routes.ts` | Tasks, semesters, calendar, weather, ticker, news, Spotify, files, settings, scholarships, contacts, notes, degree tracking, OneDrive browsing, Outlook sync, feedback |

| `catWashRoutes.ts` | All `/api/webhook/*` endpoints, `/api/shower/*`, `/api/media/*`, `/api/kitchen/*`, `/api/tts/*`, `/api/partner-status`, `/api/tablet-command/*`, HA health monitor, playback state, TTS engine, toothbrush polling, voice commands |

Optional: If you want to go further

You could also split out:

- `server/spotifyRoutes.ts` — All the `/api/spotify/*` endpoints (~900 lines)
- `server/tickerRoutes.ts` — Weather, news, pollen, ticker endpoints (~500 lines)
- `server/outlookRoutes.ts` — Outlook sync and email filing endpoints (~200 lines)

Same process — create the file, move the routes, add an import and call in `routes.ts`.

If Something Goes Wrong

Don't panic. You still have the original `routes.ts` with everything in it. If the split causes issues you can't fix:

1. Delete `catWashRoutes.ts`
2. Undo your changes to `routes.ts` (Ctrl+Z, or re-download from Replit/GitHub)
3. You're back to where you started

Nothing is lost. The split is purely organizational — no data, no database, no settings are affected.

What Was Missing — Additional Setup Details

This guide covers everything that was NOT included in the previous 4 emails. These are the gaps I found after auditing the entire codebase.

1. Google Apps Script — The Email Automation Brain

This was completely missing from all previous guides.

Your app has a unified email intake system powered by a Google Apps Script running on a separate Gmail account (homeworkbryn@gmail.com). This script monitors incoming emails and forwards them to your app via webhooks. Without it, these features stop working:

- **Email-to-ticker:** Send an email with subject starting with "Ticker" to add items to the HA news ticker
- **Email-to-reminder:** Send an email with subject starting with "Reminder" to create a task
- **Email-to-delete:** Send an email with subject starting with "Delete" to remove ticker items, calendar events, or tasks
- **Email-to-homework:** Any other email gets parsed as a homework task

How It Works

The Apps Script runs on a timer (every 5-10 minutes) in Google Apps Script, checks for new emails, and POSTs them to your app:

```
POST /api/webhook/email  
  
Body: { emailId, subject, body, from, auth: "5747" }
```

The app then routes the email based on the subject prefix to the correct handler.

What You Need to Set Up

1. **Go to script.google.com** and create a new project
2. **Write (or paste) the Apps Script** that:
 - Reads unread emails from the target Gmail inbox
 - Extracts subject, body, sender, and email ID
 - POSTs to `http://YOUR_PI_IP:5000/api/webhook/email`
 - Marks the email as read after processing
3. **Set up a time-driven trigger** to run every 5 minutes

Example Apps Script Code

```
function processEmails() {  
  var threads = GmailApp.search('is:unread', 0, 10);  
  
  var appUrl = 'http://YOUR_PI_IP:5000'; // Change to your Pi's address  
  
  for (var i = 0; i < threads.length; i++) {  
  
    var messages = threads[i].getMessages();  
  
    for (var j = 0; j < messages.length; j++) {  
  
      var msg = messages[j];  
  
      if (msg.isUnread()) {  
  
        var payload = {  
  
          emailId: msg.getId(),  
  
          subject: msg.getSubject(),  
  
          body: msg.getPlainBody(),  
  
          from: msg.getFrom(),  
  
          auth: '5747'  
  
        };  
  
        try {  
  
          UrlFetchApp.fetch(appUrl + '/api/webhook/email', {  
  
            method: 'post',  
  
            contentType: 'application/json',  
  
            payload: JSON.stringify(payload),  

```

```

        muteHttpExceptions: true

    });

    msg.markRead();

} catch (e) {

    Logger.log('Error processing email: ' + e.message);

}

}

}

}

}

```

Important: Network Access

Google Apps Script runs in Google's cloud. It CANNOT reach a device on your home network directly. You need one of these:

- **Cloudflare Tunnel** (see the Troubleshooting Guide, Section 12)
- **Port forwarding** on your router (less secure)
- **Keep using Replit** just for this webhook (the simplest option)

The Auth Code

The webhook uses a hardcoded auth code: `5747` . This is NOT a secure authentication method — it's a simple shared secret. For self-hosting, you should:

1. Change `5747` to a strong random string in both the Apps Script and `server/routes.ts`
2. Search for `auth !== '5747'` in `server/routes.ts` and replace all instances

2. TMU Academic Calendar (iCal Feed)

Missing from all guides.

The app can import academic dates (reading week, exam periods, holidays) from TMU's public iCal feed.

Environment Variable

```
TMU_ICAL_URL=https://www.torontomu.ca/content/dam/registrar/PDFs/AcademicCalendar/Academic_Calendar_2026_ical.ics
```

This is just a public URL — no authentication needed. The app fetches it periodically and parses the iCal format. If you don't set this variable, the TMU calendar features simply won't show any data (no crash).

3. OpenAI Base URL Override

Missing from Integration Guide.

On Replit, the OpenAI API calls go through Replit's AI integration proxy. On the Pi, they need to go directly to OpenAI.

The code references TWO environment variables for OpenAI:

```
AI_INTEGRATIONS_OPENAI_API_KEY=sk-xxx    # Currently used (Replit name)
AI_INTEGRATIONS_OPENAI_BASE_URL=          # Replit proxy URL
```

For self-hosting, you need to change this in **5 places** (not just the one mentioned in the Integration Guide):

| File | Line to Find | Change To |

|-----|-----|-----|

```
| server/replit_integrations/audio/client.ts | apiKey:
process.env.AI_INTEGRATIONS_OPENAI_API_KEY | apiKey: process.env.OPENAI_API_KEY |
| server/replit_integrations/audio/client.ts | baseUrl:
process.env.AI_INTEGRATIONS_OPENAI_BASE_URL | DELETE this line (use OpenAI's default URL) |
| server/replit_integrations/image/client.ts | same two lines | same changes |
| server/replit_integrations/chat/routes.ts | same two lines | same changes |
| server/routes.ts (2 places, around lines 6297 and 15272) | same two lines | same changes |
```

After removing the `baseUrl` line, OpenAI calls will go directly to `https://api.openai.com` (the default).

4. Object Storage — Detailed Replacement Plan

The Integration Guide mentioned 3 options but didn't give implementation details.

Your app stores files in two directories inside Replit Object Storage:

- `public/` — TTS audio files that the Nest speaker streams

- .private/ — uploaded PDFs

The Simplest Replacement (Local File System)

Step 1: Create the directories on the Pi:

```
mkdir -p /opt/dashboard/uploads/public
mkdir -p /opt/dashboard/uploads/private
```

Step 2: Set environment variables:

```
PUBLIC_OBJECT_SEARCH_PATHS=/opt/dashboard/uploads/public
PRIVATE_OBJECT_DIR=/opt/dashboard/uploads/private
```

Step 3: Replace the object storage client. Create a new file `server/localFileStorage.ts` :

```
import fs from 'fs';
import path from 'path';

const PUBLIC_DIR = process.env.PUBLIC_OBJECT_SEARCH_PATHS || '/opt/dashboard/uploads/public';

const PRIVATE_DIR = process.env.PRIVATE_OBJECT_DIR || '/opt/dashboard/uploads/private';

export const localStorageClient = {

  async uploadFile(filePath: string, content: Buffer, contentType?: string): Promise<void> {

    const fullPath = filePath.startsWith('.private')

      ? path.join(PRIVATE_DIR, filePath.replace(/^\.private\/?/, ''))

      : path.join(PUBLIC_DIR, filePath.replace(/^public\/?/, ''));

    const dir = path.dirname(fullPath);

    fs.mkdirSync(dir, { recursive: true });

    fs.writeFileSync(fullPath, content);

  },
```

```

async downloadFile(filePath: string): Promise<Buffer> {

    const fullPath = filePath.startsWith('.private')

        ? path.join(PRIVATE_DIR, filePath.replace(/^\.private\/?/, ''))

        : path.join(PUBLIC_DIR, filePath.replace(/^public\/?/, ''));

    return fs.readFileSync(fullPath);

},

```

```

async deleteFile(filePath: string): Promise<void> {

    const fullPath = filePath.startsWith('.private')

        ? path.join(PRIVATE_DIR, filePath.replace(/^\.private\/?/, ''))

        : path.join(PUBLIC_DIR, filePath.replace(/^public\/?/, ''));

    if (fs.existsSync(fullPath)) fs.unlinkSync(fullPath);

},

```

```

async fileExists(filePath: string): Promise<boolean> {

    const fullPath = filePath.startsWith('.private')

        ? path.join(PRIVATE_DIR, filePath.replace(/^\.private\/?/, ''))

        : path.join(PUBLIC_DIR, filePath.replace(/^public\/?/, ''));

    return fs.existsSync(fullPath);

},

```

```

async listFiles(prefix: string): Promise<string[]> {

    const dir = prefix.startsWith('.private')

```

```

    ? path.join(PRIVATE_DIR, prefix.replace(/^\.private\/?/, ''))

    : path.join(PUBLIC_DIR, prefix.replace(/^public\/?/, ''));

if (!fs.existsSync(dir)) return [];

return fs.readdirSync(dir).map(f => path.join(prefix, f));

}

};

```

Step 4: Replace imports. Everywhere the code imports from `objectStorage` , change it:

```

// OLD:

import { objectStorageClient } from "../replit_integrations/object_storage/objectStorage";

// NEW:

import { localStorageClient as objectStorageClient } from "../localFileStorage";

```

Step 5: Add static serving for the public directory. In `server/index.ts` , add:

```

app.use('/uploads', express.static('/opt/dashboard/uploads/public'));

```

This way, the Nest speaker can access TTS audio at `http://PI_IP:5000/uploads/tts-audio/filename.mp3` .

ChatGPT Prompt for Object Storage Replacement

[Paste the intro statement]

I need to replace Replit Object Storage with local file system storage. Here is my current object storage client code:

[paste server/replit_integrations/object_storage/objectStorage.ts]

And here is an example of how it's used in routes.ts:

[paste 20-30 lines showing objectStorageClient.upload or objectStorageClient.download usage]

Please create a drop-in replacement file called server/localFileStorage.ts that uses fs.readFileSync/writeFileSync instead of cloud storage. Files should go to /opt/dashboard/uploads/public (for TTS audio) and /opt/dashboard/uploads/private (for PDFs). The API should match the existing objectStorageClient exactly so I only need to change the import line.

5. Replit-Specific Environment Variables to Remove/Replace

Missing from all guides.

These environment variables are set automatically by Replit and don't exist on a Pi. The code references them and needs small tweaks:

Variable	Used For	What to Do on Pi
REPLIT_CONNECTORS_HOSTNAME	OAuth connector API	Not needed — you're using direct OAuth
REPL_IDENTITY	Internal auth token	Not needed
WEB_REPL_RENEWAL	Deployment auth token	Not needed
REPLIT_DEV_DOMAIN	Dev server URL	Replace with Pi hostname
REPLIT_DOMAINS	Production URL	Replace with DEPLOYED_APP_URL
REPL_SLUG	Repl name	Not needed
REPL_OWNER	Repl owner username	Not needed

The code already has fallbacks for most of these (e.g., process.env.REPLIT_DOMAINS?.split(',')[0] || 'fallback'), but you should audit server/spotify.ts and server/secondGoogleAccount.ts where REPLIT_DOMAINS is used for building redirect URIs. Make sure those fallbacks point to your Pi's address.

6. Self-Ping Keep-Alive

Not documented anywhere.

The app pings itself every 4 minutes to stay alive on Replit (which sleeps idle apps). On a Pi with systemd, this is unnecessary but harmless. The code is around line 7610 in `routes.ts` :

```
const SELF_PING_INTERVAL_MS = 4 * 60 * 1000;

setInterval(async () => {

    await fetch( http://localhost:${process.env.PORT || 5000}/api/version );

}, SELF_PING_INTERVAL_MS);
```

You can leave this in (it doesn't hurt) or remove it to keep logs cleaner.

7. Background Scheduled Tasks — Full List

The Self-Hosting Guide mentioned background processes but didn't list them all.

Here's every scheduled process running inside the app:

Process	Interval	What It Does	Where in Code
-----	-----	-----	-----
Reminder Scheduler	Every 60 seconds	Checks for due task reminders, sends Echo voice announcements + push notifications via Resend email	<code>server/reminderScheduler.ts</code>
Semester Auto-Activation	Every 6 hours + startup	Activates the semester whose date range includes today	<code>routes.ts</code> ~line 7607
Audio Pre-Generation	Every 30 minutes	Pre-generates TTS audio for upcoming unlistened files	<code>routes.ts</code> ~line 8100
HA Health Check	Every 60 seconds	Pings HA to track connectivity status	<code>routes.ts</code> ~line 7527
Toothbrush Polling	Every 3 seconds (only during playback)	Checks if brushing has started, auto-stops reading	<code>routes.ts</code> ~line 8683
Word Advancement	Every ~50ms (only during playback)	Advances the highlighted word on tablet/TV display	<code>routes.ts</code> ~line 8762
Self-Ping	Every 4 minutes	Keep-alive for Replit (unnecessary on Pi)	<code>routes.ts</code> ~line 7613
Tablet Command Polling	Every 3 seconds	The tablet polls for new navigation/playback commands	<code>routes.ts</code> ~line 998

These all start automatically when the app boots — no configuration needed.

8. The Webhook Auth Code (5747)

Security issue not documented.

Multiple webhook endpoints use a hardcoded auth code 5747 :

- /api/webhook/email — the Gmail intake
- /api/webhook/reminder — email-to-task
- /api/webhook/delete — email-to-delete
- /api/webhook/email-homework — email-to-homework (uses X-Webhook-Secret header)

For self-hosting, you should:

1. Pick a strong random string (e.g., generate with `openssl rand -hex 16`)
2. Search for '5747' in `server/routes.ts` and replace all instances
3. Update the matching value in your Google Apps Script
4. Add a new env var instead of hardcoding:

```
WEBHOOK_AUTH_SECRET=your_random_string_here
```

9. The Replit AI Integrations System

Missing from all guides.

Your app uses Replit's AI Integrations for more than just TTS. It's used for:

1. **TTS Audio Generation** (OpenAI TTS voices) — used by the cat washroom reading system
2. **Chat Completions** — used by the syllabus parser (`/api/syllabus/parse`) and assignment PDF parser (`/api/tasks/parse-assignment-pdf`)
3. **Image Generation** — available but may not be actively used

All of these use `AI_INTEGRATIONS_OPENAI_API_KEY` and `AI_INTEGRATIONS_OPENAI_BASE_URL` .

Full List of Files to Update

<code>server/replit_integrations/audio/client.ts</code>	— TTS generation
<code>server/replit_integrations/image/client.ts</code>	— Image generation
<code>server/replit_integrations/chat/routes.ts</code>	— Chat completions
<code>server/routes.ts</code> (line ~6297)	— Syllabus parser
<code>server/routes.ts</code> (line ~15272)	— Assignment PDF parser
<code>server/routes.ts</code> (line ~15387)	— Course list comparison

In EACH of these files, change:

```
// FROM:

apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,

baseUrl: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,

// TO:

apiKey: process.env.OPENAI_API_KEY,

// (delete the baseUrl line entirely)
```

10. Uppy File Upload Widget

Not documented.

The app uses Uppy (a file upload library) for uploading PDFs. On Replit, Uppy uploads go to Replit's Object Storage via the `@uppy/aws-s3` plugin. On the Pi, you'd need to:

1. Replace the Uppy S3 plugin with direct upload (Uppy supports XHR upload out of the box)
2. Or keep using the current upload endpoints which already handle the file saving

The upload endpoint is at `/api/course-week-upload` in `routes.ts`. It receives the file, stores it via Object Storage, and creates a database record. Once you replace Object Storage with local files (see section 4 above), uploads will work with local storage automatically.

11. The `.spotify-token.json` File

Not documented.

Spotify tokens are stored in a file called `.spotify-token.json` in the app root directory. This file is auto-created when you authorize Spotify via `/api/spotify/login`. On the Pi:

- Make sure the app has write access to its directory: `sudo chown -R pi:pi /opt/dashboard`
 - The file will be created automatically after the first Spotify login
 - If you copy the app from Replit, you can also copy this file to skip re-authorization (but the tokens may have expired)
-

12. Replit Vite Plugins (Dev Dependencies)

Not documented.

The dev dependencies include three Replit-specific Vite plugins:

```
@replit/vite-plugin-cartographer
```

```
@replit/vite-plugin-dev-banner
```

```
@replit/vite-plugin-runtime-error-modal
```

These are only used during development. The production build (`npm run build`) should work without them, but if you get build errors related to these plugins on the Pi:

1. Open `vite.config.ts`
2. Find the plugin imports and remove them
3. Remove them from the `plugins: [...]` array
4. Or just comment them out with `//`

The app will build and run fine without them.

13. Session Storage (connect-pg-simple)

Not documented.

The app uses `connect-pg-simple` to store user sessions (login state) in PostgreSQL. This is already configured and will work on the Pi with no changes, as long as your `DATABASE_URL` is correct.

If you see errors about a `session` table not existing:

The table is auto-created, but if it's not:

```
sudo -u postgres psql -d dashboard_db -c "  
  
CREATE TABLE IF NOT EXISTS session (  
  
    sid VARCHAR NOT NULL PRIMARY KEY,  
  
    sess JSON NOT NULL,  
  
    expire TIMESTAMP(6) NOT NULL  
  
);  
  
CREATE INDEX IF NOT EXISTS IDX_session_expire ON session (expire);  
  
"
```

14. Edge TTS (Node Package vs Python Package)

Partially documented but confusing.

The app uses TWO different edge-tts implementations:

1. **Python edge-tts** — installed via `pip3 install edge-tts` (the fallback TTS engine)
2. **Node.js node-edge-tts** — installed via `npm` as a dependency in `package.json`

Both should work on the Pi. The Python version is used as a command-line fallback. The Node version is used directly from the server code. Make sure both are installed:

Python version (already in the setup guide)

```
pip3 install edge-tts --break-system-packages
```

Node version (installed automatically with npm install)

```
npm install
```

Summary: Complete Checklist of Things to Set Up

Use this as your master checklist. Check off each item as you complete it:

- ☐ 1. Flash Pi OS and SSH in
- ☐ 2. Install Node.js 20, PostgreSQL, espeak-ng, edge-tts (Python)
- ☐ 3. Create database user and database
- ☐ 4. Transfer code to Pi
- ☐ 5. Run npm install
- ☐ 6. Create .env file with ALL variables (use the template from Integration Guide)
- ☐ 7. Rewrite getAccessToken() in googleCalendar.ts (Replit connector to direct OAuth)
- ☐ 8. Rewrite getAccessToken() in gmail.ts (same)
- ☐ 9. Rewrite getAccessToken() in onedrive.ts (same)
- ☐ 10. Rewrite getOutlookAccessToken() in outlookCalendar.ts (same)

- [] 11. Change OpenAI env vars in 5 files (AI_INTEGRATIONS_ to OPENAI_)
- [] 12. Remove baseURL lines for OpenAI in those same 5 files
- [] 13. Fix Spotify redirect URI in spotify.ts (REPLIT_DOMAINS to DEPLOYED_APP_URL)
- [] 14. Fix redirect URI in secondGoogleAccount.ts (same)
- [] 15. Fix redirect URI in thirdGoogleAccount.ts (same)
- [] 16. Replace Object Storage with local file system (or MinIO)
- [] 17. Replace webhook auth code 5747 with a strong secret
- [] 18. Remove or comment out Replit Vite plugins if build fails
- [] 19. Set up Google OAuth and get refresh tokens
- [] 20. Set up Azure app and get Microsoft refresh token
- [] 21. Authorize Spotify via /api/spotify/login
- [] 22. Set up Resend account and get API key
- [] 23. Set up OpenAI account and get API key
- [] 24. Set up Google Apps Script for email intake (if needed)
- [] 25. Run npm run db:push to create database tables
- [] 26. Run npm run build
- [] 27. Set up systemd service
- [] 28. Update HA rest_command URLs to point to Pi
- [] 29. Test each integration one by one
- [] 30. Set up Cloudflare Tunnel if you need external access

Web Dev, Python, Bash & Grep

A Complete Beginner's Guide for Dummies

HTML (HyperText Markup Language) is the backbone of every website. Think of it like the skeleton of a body — it gives structure to web pages. Every website you've ever visited is built on HTML.

When you visit a website, your browser (Chrome, Firefox, Safari) downloads an HTML file and reads it like instructions. The HTML tells the browser: "Put a heading here, a paragraph there, an image over here." It doesn't make things pretty or interactive — it just defines what's on the page.

— HTML uses "tags" to define content. Tags are words wrapped in angle brackets. Most tags come in pairs: an opening tag and a closing tag (with a /).

```
<h1>This is a big heading</h1>
```

```
<p>This is a paragraph of text.</p>
```

```

```

□ — A tag plus its content equals an "element." So `<p>Hello</p>` is a paragraph element. — Extra information you attach to a tag. They go inside the opening tag.

```

```

```
<a href="https://google.com" target="_blank">Open Google</a>
```

— Every HTML page follows this basic structure:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>My Page Title</title>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome!</h1>
```

```
<p>This is my first web page.</p>
```

```
</body>
```

```
</html>
```

□ — Tags go inside other tags, like folders inside folders. The inner tag is a "child" of the outer tag.

```
<div>
```

```
<h2>My Section</h2>
```

```
<p>Some text inside the section.</p>
```

```
</div>
```

The `<h2>` and `<p>` are children of the `<div>` . This parent-child relationship is how you organize

a page.

A `<div>` is like an invisible box you use to group content together. Think of it like a shipping container — you can put anything inside it and move it around as one unit.

```
<div class="product-card">

<h2>Running Shoe</h2>
<p class="price">$89.99</p>
<button>Add to Cart</button>
</div>
```

Think: `div` = "division" of the page. Each `div` is a section you can style and control independently.

□ — When you write `class="product-card"`, you're giving that `div` a name. CSS and JavaScript use this name to find and style/control it. One class can be used on many elements. — Like `class`, but unique. Only one element on the page should have a given `id`. Use it when you need to target one specific thing.

```
<div id="main-header">This is the one and only header</div>
<div class="card">Card 1</div>
<div class="card">Card 2</div>
<div class="card">Card 3</div>
```

CSS (Cascading Style Sheets) controls how HTML looks — colors, sizes, fonts, spacing, layouts. If HTML is the skeleton, CSS is the skin and clothes.

You write "rules" that say "find this element and make it look like this."

```
.product-card {
background-color: white;
border-radius: 8px;
padding: 20px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1);
width: 300px;
}
```

This says: "Find every element with the class `product-card` and give it a white background, rounded corners, inner spacing, a subtle shadow, and make it 300 pixels wide."

```
□ h1 { }
/* All <h1> tags */
.card { }
/* All elements with class="card" */
```



```

#main-header { }

/* The one element with id="main-header" */

div p { }

/* All <p> tags inside a <div> */

.card:hover { }

/* A .card when the mouse hovers over it */

color: #333333;

/* Text color (dark gray) */

background-color: #fofofo;

/* Background color (light gray) */

background-color: rgb(255, 0, 0);

/* Red using RGB */

background-color: rgba(0, 0, 0, 0.5); /* Black at 50% opacity */

width: 300px;

/* Fixed width in pixels */

width: 50%;

/* Half the parent's width */

height: 200px;

/* Fixed height */

max-width: 1200px;

/* Won't grow beyond this */

min-height: 100vh;

/* At least the full screen height */

padding: 20px;

/* Space INSIDE the box (between border and content) */

margin: 10px;

/* Space OUTSIDE the box (between this box and neighbors) */

padding: 10px 20px;

/* 10px top/bottom, 20px left/right */

margin: 0 auto;

/* Center the element horizontally */

□Think of it like a picture frame: is the matting inside the frame, is the wall space around the frame.

font-size: 16px;

/* Text size */

```

```
font-weight: bold;

/* Bold text (or 700) */

font-family: Arial, sans-serif;

/* Font choice */

text-align: center;

/* Center the text */

line-height: 1.5;

/* Space between lines */

border: 1px solid #ccc;

/* 1px gray border */

border-radius: 8px;

/* Rounded corners */

border-bottom: 2px solid blue; /* Only bottom border */

.container {

display: flex;

/* Activate flexbox */

justify-content: center; /* Center items horizontally */

}

align-items: center;

/* Center items vertically */

gap: 20px;

/* Space between items */

flex-direction: row;

/* Items in a row (default) */

flex-wrap: wrap;

/* Wrap to next line if no room */
```

□ Flexbox is the most important layout tool in modern CSS. Think of it like saying "put these items in a row (or column) and here's how to space them out."

— Making your page look good on any screen size:

```
@media (max-width: 768px) {

.container {

flex-direction: column;

/* Stack vertically on small screens */

}
```

```
.card {  
width: 100%;  
  
/* Full width on mobile */  
}  
}
```

JavaScript (JS) makes web pages interactive. It's the brain that responds to clicks, updates content, fetches data, and controls behavior. Every modern website uses it.

JavaScript runs in the browser. When you visit a website, the browser downloads the HTML, CSS, and JavaScript files. The JS code then runs on your computer, making the page interactive without needing to reload.

— Containers that store data.

```
let name = "Bryn";  
  
// Can be changed later  
  
const age = 20;  
  
// Cannot be changed (constant)  
  
let score = 0;  
  
// A number  
  
□let isLoggedIn = true;  
  
// A boolean (true or false)
```

— Reusable blocks of code. Like a recipe you can follow over and over.

```
function greet(name) {  
return "Hello, " + name + "!";  
}  
  
greet("Bryn");  
  
// Returns "Hello, Bryn!"  
  
greet("Alex");  
  
// Returns "Hello, Alex!"  
  
// Arrow function (shorter syntax, same thing)  
  
const add = (a, b) => a + b;  
  
add(5, 3);  
  
// Returns 8
```

— Making decisions.

```
let temperature = 30;  
  
if (temperature > 25) {
```

```
console.log("It's hot outside!");  
} else if (temperature > 15) {  
  console.log("It's nice outside.");  
} else {  
  console.log("It's cold outside!");  
}
```

— Ordered lists of things.

```
let fruits = ["apple", "banana", "cherry"];  
fruits[0];  
// "apple" (first item, counting starts at 0)  
fruits.length;  
// 3  
fruits.push("date");  
// Add to the end  
fruits.includes("banana");  
// true  
// Loop through every item  
fruits.forEach(fruit => {  
  console.log(fruit);  
});  
// Transform every item  
let upperFruits = fruits.map(f => f.toUpperCase());  
// ["APPLE", "BANANA", "CHERRY", "DATE"]  
// Filter items  
let longNames = fruits.filter(f => f.length > 5);  
// ["banana", "cherry"]  
— Collections of key-value pairs (like a dictionary).  
let student = {  
  name: "Bryn",  
  age: 20,  
  major: "Computer Science",  
  grades: [85, 92, 78]  
};  
student.name;
```

```

// "Bryn"
student.grades[0];

// 85
student.major = "Math"; // Change a value

— Reacting to user actions (clicks, typing, scrolling).
const button = document.getElementById("myBtn");
button.addEventListener("click", () => {
  document.getElementById("output").textContent = "You clicked!";
});

— JavaScript can change the HTML page on the fly.
// Find an element
const heading = document.querySelector("h1");
□// Change its text
heading.textContent = "New Title!";
// Change its style
heading.style.color = "blue";
heading.style.fontSize = "48px";
// Add a new element
const newParagraph = document.createElement("p");
newParagraph.textContent = "I was added by JavaScript!";
document.body.appendChild(newParagraph);
// Remove an element
heading.remove();
// Get data from an API
const response = await fetch("https://api.example.com/tasks");
const tasks = await response.json();
console.log(tasks);
// Send data to a server
await fetch("/api/tasks", {
  □method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Do homework", dueDate: "2026-04-01" })
});

```

HTML = the house structure, CSS = the paint and decor, JavaScript = the electricity and plumbing

that makes things work.

Python is a general-purpose programming language known for being easy to read and learn. It's used for web backends, data science, AI, automation, and more. Many universities teach it as a first language because its syntax reads almost like English.

JavaScript runs in the browser (frontend). Python typically runs on a server or your computer (backend). You can build websites with both, but they're used for different parts.

— Python is simpler than JavaScript. No let or const needed.

```
name = "Bryn"
```

```
age = 20
```

```
pi = 3.14159
```

```
is_student = True
```

Note: True/False are capitalized in Python

— Python uses spaces (usually 4) instead of { } braces to define code blocks. This is the biggest gotcha for beginners.

Correct (4 spaces of indentation)

```
if age >= 18:
```

```
    print("You're an adult")
```

```
print("You can vote")
```

```
if age >= 18:
```

```
    print("You're an adult")
```

Not indented = broken

— Like JavaScript arrays.

```
fruits = ["apple", "banana", "cherry"]
```

```
fruits[0]
```

"apple"

```
len(fruits)
```

```
fruits.append("date")
```

Add to end

```
for fruit in fruits:
```

```
    print(fruit)
```

```
upper_fruits = [f.upper() for f in fruits]
```

```
long_names = [f for f in fruits if len(f) > 5]
```

— Like JavaScript objects (key-value pairs).

```
student = {
```

```
    "name": "Bryn",
```

```
    "age": 20,
```

```
    "major": "Computer Science"
```

```
}
```

```
student["name"]
```

"Bryn"

```
student["gpa"] = 3.8
```

Add new key

```
def calculate_average(grades):
```

```
    total = sum(grades)
```

```
    return total / len(grades)
```

```
my_grades = [85, 92, 78, 95, 88]
```

```
avg = calculate_average(my_grades)
```

```
print(f"Your average is {avg}")
```

□ — Python's way of putting variables inside text (the f before the quotes):

```
name = "Bryn"
```

```
age = 20
```

```
print(f"My name is {name} and I'm {age} years old.")
```

```
score = 85

if score >= 90:

    grade = "A"

elif score >= 80:

    grade = "B"

elif score >= 70:

    grade = "C"

else:

    grade = "F"

print(f"Your grade is {grade}")
```

"Your grade is B"

— Python has thousands of add-on packages:

```
□import math

print(math.sqrt(144))
```

12.0

```
import random

print(random.randint(1, 100))
```

Random number between 1 and 100

```
from datetime import datetime

now = datetime.now()

print(now.strftime("%B %d, %Y"))
```

Write to a file

```
with open("notes.txt", "w") as f:

    f.write("Hello, world!\n")

    f.write("Second line.\n")

with open("notes.txt", "r") as f:
```



```
content = f.read()
```

```
print(content)
```

"March 28, 2026"

□ Bash is the language of the terminal (also called the command line, shell, or console). It's how you talk directly to your computer without a graphical interface. Every Mac, Linux computer, and server uses it. On Windows, you can use Git Bash or WSL.

```
pwd
```

Print Working Directory - shows where you are

Example output: /home/bryn/Documents

```
ls
```

List - shows files and folders in current directory

```
ls -l
```

Long format - shows sizes, dates, permissions

```
ls -la
```

Also show hidden files (ones starting with .)

```
ls -lh
```

Human-readable file sizes (KB, MB, GB instead of bytes)

```
cd Documents
```

Change Directory - move into the Documents folder

```
cd ..
```

Go up one level (back to parent folder)

```
cd ~
```

Go to your home directory

```
cd /
```

Go to the root of the entire filesystem

Think of `cd` like double-clicking a folder, and `ls` like looking at what's inside.

```
mkdir my-project
```

Make a new directory (folder)

```
mkdir -p a/b/c
```

Make nested folders all at once

```
touch notes.txt
```

Create an empty file (or update its timestamp)

```
cp file.txt backup.txt
```

Copy a file

```
cp -r my-folder backup-folder
```

Copy a folder and everything in it

```
mv old-name.txt new-name.txt
```

Rename a file

```
mv file.txt ~/Desktop/
```

Move a file to the Desktop

```
rm file.txt
```

Delete a file (PERMANENT - no recycle bin!)

```
rm -r folder/
```

Delete a folder and everything in it

```
rm -rf folder/
```

Force delete - no confirmation prompts (DANGEROUS)

rm is permanent. There is no undo. Always double-check before deleting.

```
cat file.txt
```

Print the entire file to the screen

```
head file.txt
```

Show the first 10 lines

```
head -n 20 file.txt
```

Show the first 20 lines

`tail file.txt`

Show the last 10 lines

`tail -f log.txt`

Follow a file live (great for watching logs)

`less file.txt`

View a file page by page (press q to quit)

`wc -l file.txt`

Count the number of lines in a file

The pipe `|` takes the output of one command and feeds it into another. This is one of Bash's superpowers.

`ls -la | head -5`

List files, but only show the first 5 lines

`cat file.txt | wc -l`

Count lines in a file

`history | tail -20`

Show your last 20 commands

`echo "Hello" > file.txt`

Write "Hello" to file (overwrites!)

```
echo "World" >> file.txt
```

Append "World" to file (adds to end)

Think of | like a conveyor belt: the output of the left side slides into the input of the right side.

```
clear
```

Clear the terminal screen

```
history
```

Show all your past commands

```
whoami
```

Show your username

```
date
```

Show current date and time

```
df -h
```

Show disk space usage

```
du -sh folder/
```

Show total size of a folder

```
top
```

Live view of running processes (q to quit)

kill 12345

Stop a process by its ID number

chmod 755 script.sh

Change file permissions (make it executable)

node app.js

Run a JavaScript file with Node.js

python3 script.py

Run a Python script

npm install

Install JavaScript packages

☐ npm run dev

Run a development server

pip install requests

Install a Python package

echo \$HOME

Print the value of the HOME variable

```
export MY_VAR="hello"
```

Set a variable for this session

```
echo $MY_VAR
```

Print it

```
DATABASE_URL=postgresql://user:pass@localhost/mydb
```

```
mkdir project && cd project
```

Do both - only cd if mkdir succeeds

```
npm install && npm run build
```

Install, then build

```
command1 ; command2
```

Run both regardless of success

```
npm run dev &
```

Run in background (& at the end)

Grep stands for global regular expression print. It searches through files for lines that match a pattern.

Think of it as Ctrl+F for your entire computer.

```
❏ grep "error" log.txt
```

This searches log.txt and prints every line containing the word "error."

```
grep "error" log.txt
```

Find lines with "error"

```
grep -i "error" log.txt
```

Case-insensitive (finds Error, ERROR, error)

```
grep -n "error" log.txt
```

Show line numbers

```
grep -c "error" log.txt
```

Count matching lines

```
grep -r "TODO" .
```

Search recursively in all files under current

folder

```
grep -l "error" *.txt
```

Only show filenames that match (not the lines)

```
grep -v "debug" log.txt
```

Invert - show lines that DON'T match

```
grep -w "cat" file.txt
```

Whole word only (won't match "catalog")

```
grep "function" *.js
```

Search all JavaScript files in current folder

```
grep -r "import" src/
```

Search all files in src/ folder and subfolders

```
grep -rn "TODO" --include="*.ts" .
```

Search only TypeScript files, show line

numbers

This is where grep really shines - filtering the output of other commands:

❑ # Find running Node processes

```
ps aux | grep "node"
```

```
npm list | grep "express"
```

```
history | grep "git"
```

```
ls -la | grep -i ".pdf"
```

```
cat server.log | grep -c "ERROR"
```

```
grep -rn "calculateGrade" --include="*.ts" .
```

Grep supports patterns (called "regular expressions" or "regex") for flexible searching:

❑ `grep "^Error" log.txt`

Lines that START with "Error"

```
grep "done$" log.txt
```

Lines that END with "done"

```
grep "error\|warning" log.txt
```

Lines with "error" OR "warning"

```
grep "task-[0-9]" log.txt
```

"task-" followed by any digit (task-1, task-9)

grep "2026-03-.." log.txt

Any date in March 2026

grep -E "[0-9]{3}-[0-9]{4}" contacts.txt

Phone number pattern (555-1234)

Find all TODO comments in your codebase

```
grep -rn "TODO\  
FIXME\  
HACK" --include="*.ts" --include="*.tsx" .  
grep -rn "function calculateGrade" --include="*.ts" .  
grep "ERROR" /var/log/app.log  
grep -rn "\.product-card" --include="*.css" .  
□grep -rn "app\.\(get\  
post\  
put\  
delete\)" --include="*.ts" server/  
grep -rl "export default function" --include="*.tsx" src/ | wc -l
```

Technology

What It Is

Analogy

Structure

(what's on the

page)

The

skeleton/

frame of a

house

Style (how it

looks)

The paint,
furniture,
and decor

Behavior (what
it does)

The
electricity
and
plumbing

A box to group
HTML elements
A shipping
container

Backend logic,
scripts, data, AI

A Swiss

Army knife

□ When you visit a website:

1. defines the content - headings,
paragraphs, buttons, images
2. makes it look good - colors, spacing,
responsive layout
3. makes it interactive - handles clicks,
loads data, updates the page
4. (or another backend language) runs on
the server - handles databases,
authentication, business logic
5. is how developers run, deploy, and
manage everything on the server
6. is how developers find things in
codebases and debug problems

WebDev_Python_Bash_Gr
ep_Guide.md

23 KB

Command-line

interface to

your computer

Talking

directly to

your

computer

Search engine

for file contents

Ctrl+F for

your whole

computer